



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE



Assignment 1 - 2D incompressible flow through a converging nozzle

Fazio Samuel 10843993-287241
Sgotto Andrea 10796906-286858
Ferrario Alessandro 10766865-286847

Working Group: number 15

Abstract:

This report documents the computational fluid dynamics (CFD) analysis of a two-dimensional incompressible flow through a converging nozzle. The main objective of this study is to perform a grid independence study to characterize the fluid behavior in two distinct flow regimes: laminar and turbulent. To this end, three main simulations were carried out:

- Laminar Case
- Turbulent Case with $k - \varepsilon$
- Turbulent Case with K- ω SST

The methodology is based on the use of the open-source software OpenFOAM for the numerical solution of the Reynolds-Averaged Navier-Stokes (RANS) equations that govern fluid motion. For each case study, several structured meshes were generated. The pressure-velocity coupling was handled using the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) algorithm, and various numerical schemes were employed for the discretization of the equations.

Advisor:

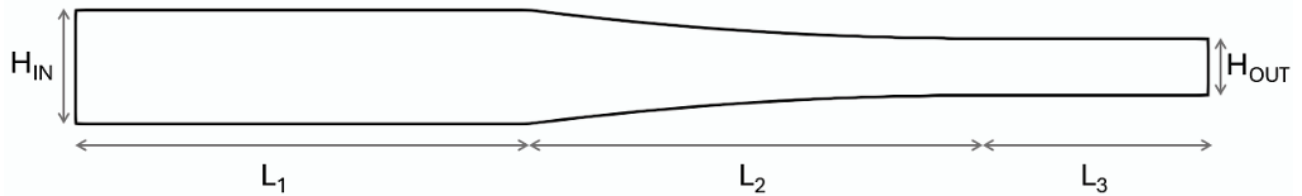
Prof. Montenegro Gianluca, Prof. Persico Giacomo

Academic Year:

2024-2025

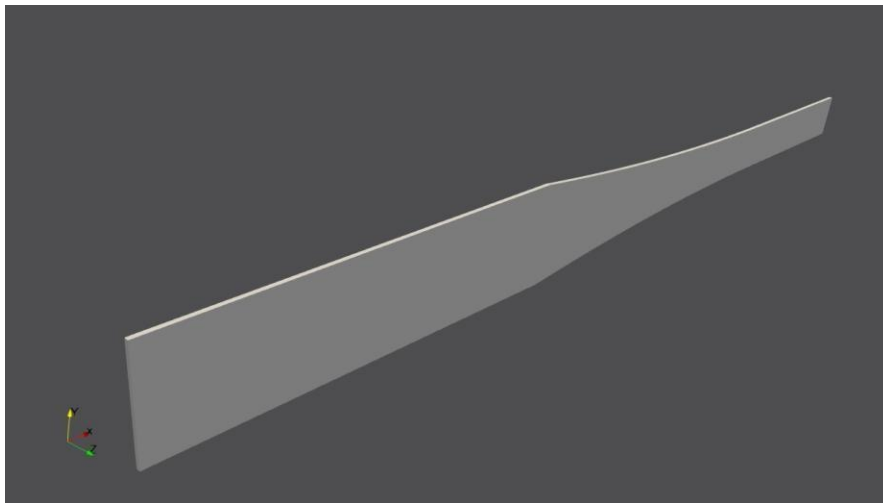
Geometrical domain

The simulations were performed on a 2D case of an incompressible flow which is running across a converging nozzle. The geometry of the domain is characterized by a rectangular inlet block and a rectangular outlet one, connected by the nozzle. The nozzle is shaped by circular arcs, which are tangent to the top and bottom lines of the outlet block.



Domain dimensions:

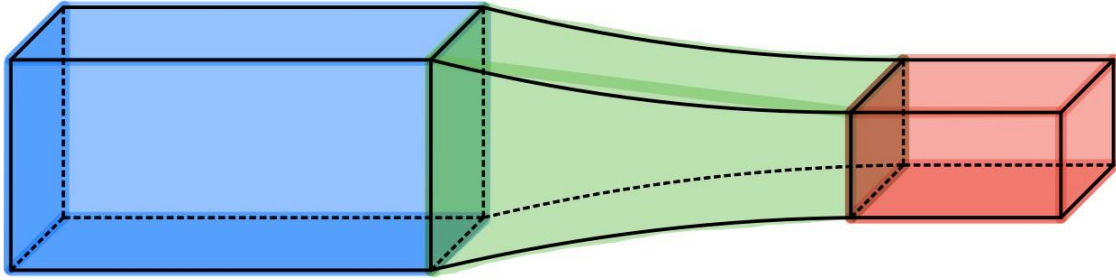
- $L_1 = 1 \text{ m}$
- $L_2 = 1 \text{ m}$
- $L_3 = 0.5 \text{ m}$
- $H_{IN} = 0.25 \text{ m}$
- $H_{OUT} = 0.125 \text{ m}$



Domain in 3D space

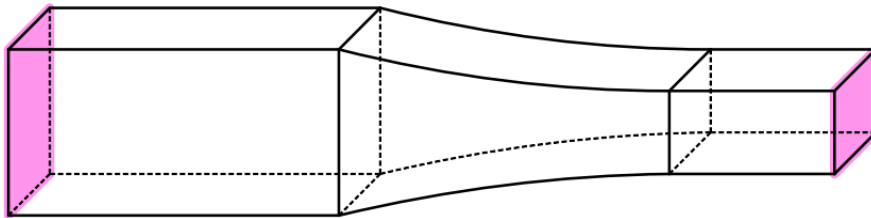
The entire domain has been divided into three different sub-blocks to help the refinement procedure of the mesh. They represent respectively the Inlet, Nozzle and Outlet regions. The fluid considered for all simulations is air at ambient conditions. It was modelled as an incompressible fluid with the following constant properties:

- Kinematic viscosity (ν): $\nu = 15 \times 10^{-6} \text{ m}^2/\text{s}$
- Density (ρ): $\rho = 1.2 \text{ kg/m}^3$

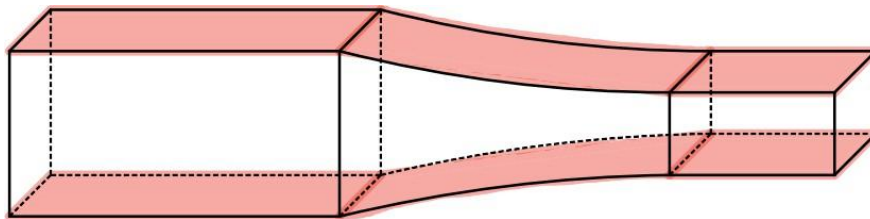


Each single block is structured hexahedral and is identified by its vertices in order to have a relative right-hand system; for what concerns the nozzle, the curved edges have been created by means of splines which guarantees the tangential constraint of the nozzle with the Outlet region.

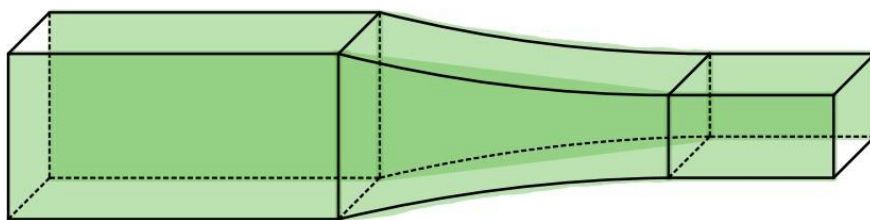
The following boundary conditions have been imposed to reproduce the simulation environment; each patch's vertices are ordered to create a surface whose normal points outwards.



Boundary type: patch



Boundary type: walls

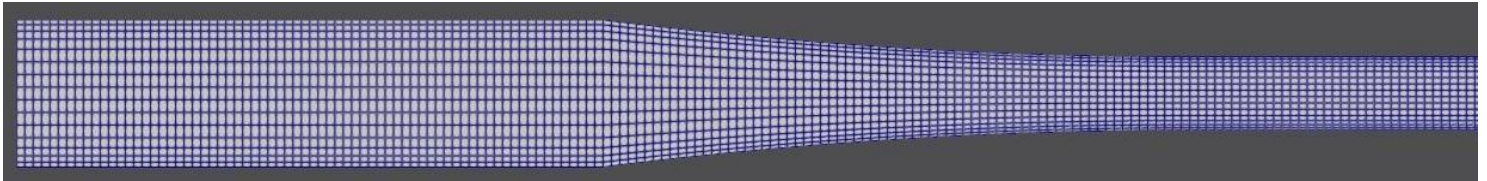


Boundary type: front/back

Meshes layout

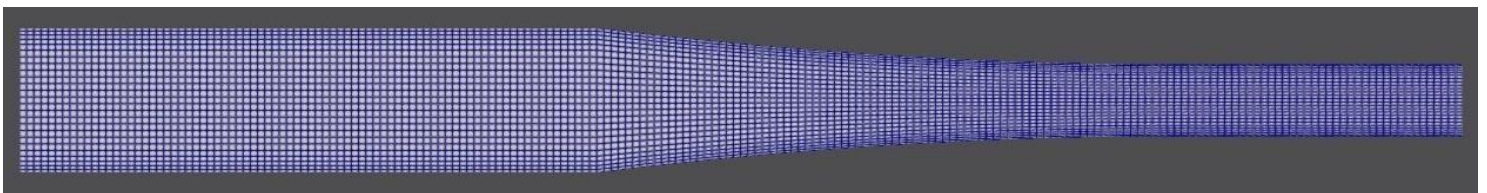
The assignment required to make a grid independence study on the fluid running across the channel in both laminar and turbulent regime. Three different simulations have been performed on three different meshes with three different levels of finishing. The number of cells in x and y directions were gradually incremented (through the simulations, with always having a constant mesh spacing in x direction, and the grading was kept unchanged.), while the grading was kept unchanged.

Coarse Mesh



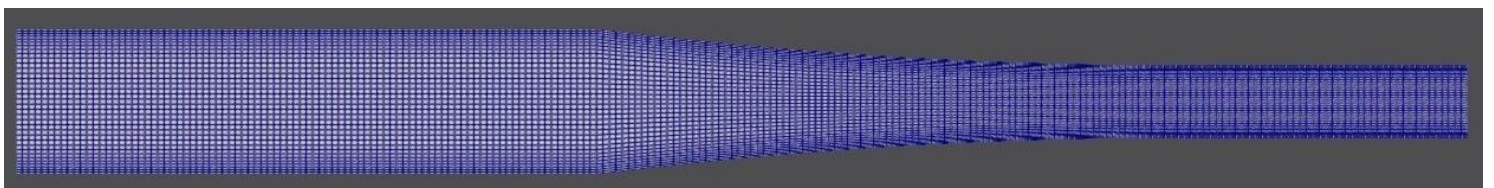
- Cells in x direction: 175
- Cells in y direction: 15

Medium mesh



- Cells in x direction: 200
- Cells in y direction: 25

Fine mesh

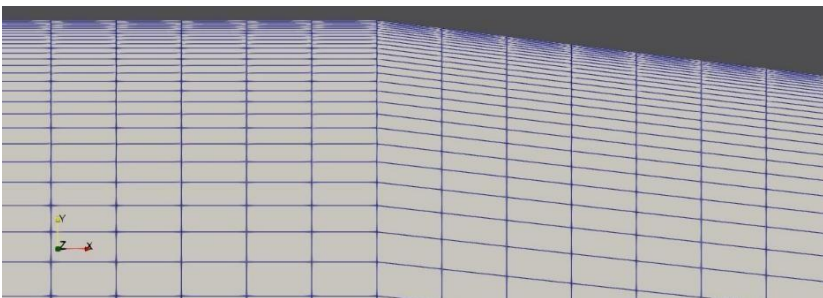
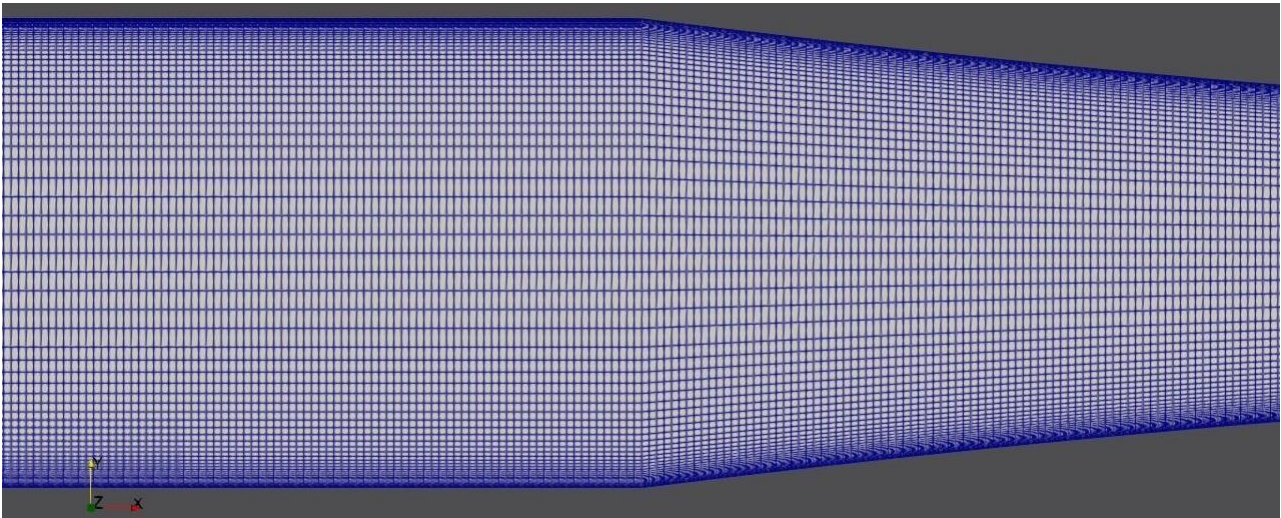


- Cells in x direction: 225
- Cells in y direction: 35

```
simpleGrading ( 1
  (
    (0.2 0.25 2)
    (0.6 0.5 1)
    (0.2 0.25 0.5)
  )
  1 )
```

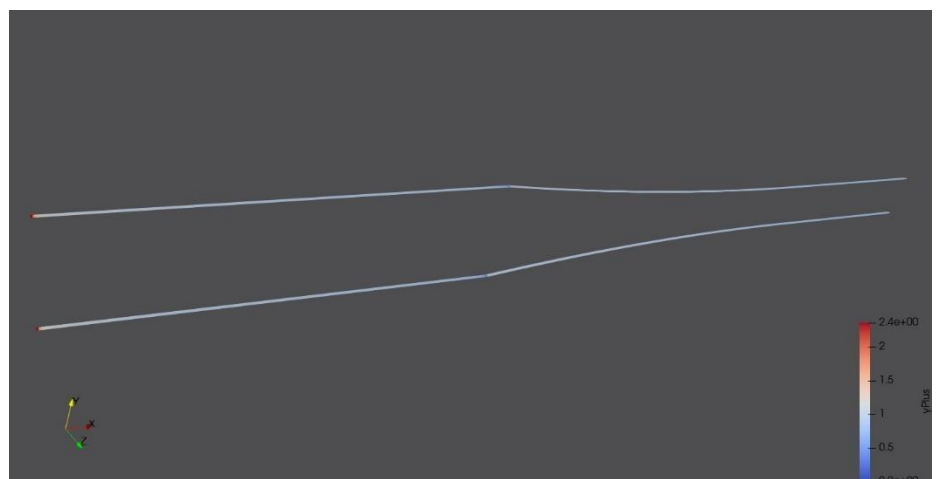
Wall mesh

For the turbulent case, it was also required to deliver a simulation without the usage of wall functions. To do so, it was necessary to create an additional mesh that was able to guarantee the correct y^+ values near the walls. This mesh shows changes in terms of number of cells but also for what concerns grading in vertical direction.



```
simpleGrading ( 1
(
(0.1 0.3333 40)
(0.2 0.1111 2)
(0.4 0.1112 1)
(0.2 0.1111 0.5)
(0.1 0.3333 0.025)
)
1 )
```

- Cells in x direction: 625
- Cells in y direction: 90



Flow model and boundary conditions

Flow model

In this case a 2D incompressible flow was simulated within a converging nozzle. It was easily verified that one of the main hypotheses of our case was feasible to reality: the assumption of incompressibility is valid due to the low flow speeds relative to the speed of sound; the estimated inlet velocity is 0.0634 m/s, and for the turbulent cases, it is 7.341 m/s. These velocities are significantly lower than the speed of sound in air (approximately 343 m/s), resulting in negligible density variations and a low Mach number. The flow regime (laminar or turbulent) is characterized by the Reynolds number (Re), calculated as $[Re = \rho V H_{in} / \mu = V H_{in} / \nu]$, where ρ is the fluid density, V is the characteristic velocity, H_{in} is the inlet height (characteristic length), and ν is the kinematic viscosity. Given $\nu = 15 \times 10^{-6} \text{ m}^2/\text{s}$ and $H_{in} = 0.25 \text{ m}$.

- For Laminar case (a) = with the small pressure difference of $[P_{T,in} - P_{out} = 0.0096 \text{ Pa}]$ applied, this results in low flow velocities and consequently a low Reynolds number, consistent with a laminar flow regime. In this case it was evaluated that $Re = 1056.7$, which is well within the typical range for laminar flow.
- For Turbulent cases (b) and (c) = the imposed pressure is a difference between inlet total pressure and outlet static pressure, so same type of condition, but with a bigger entity because now its value is 129.34 Pa. The higher estimated inlet velocity led to a value of $Re = 122350$, so a number clearly indicates a turbulent flow regime, necessitating the use of turbulence models for accurate prediction of flow phenomena.

The governing equations solved for the fluid flow are the Reynolds-Averaged Navier-Stokes (RANS) equations, which represent the conservation of mass (continuity equation) and momentum.

For the Laminar Case (a): Since the flow is assumed and confirmed to be laminar no turbulence model is required or applied. The Navier-Stokes equations are solved directly without any turbulent viscosity terms.

For the Turbulent Cases (b) and (c): Given the high Reynolds number for the other cases a turbulence model is essential to account for the effects of turbulent fluctuations. We've chosen a different model for each case, taking in account the assignment.

Case (b): it was used the standard $k-\epsilon$ Turbulence Model with wall functions. The $k-\epsilon$ model is a two-equation eddy-viscosity model that solves transport equations for two variables: the turbulent kinetic energy (k) and its dissipation rate (ϵ). Standard wall functions were used to bridge the gap between the bulk flow region and the viscous sublayer near the wall, reducing the need for extremely fine near wall meshes.

Case (c): it was used the SST $k-\omega$ Turbulence Model (Wall-Resolved). The SST (Shear Stress Transport) $k-\omega$ model is a two-equation eddy-viscosity model that combines the advantages of the $k-\omega$ model (robustness in the near-wall region) and the $k-\epsilon$ model (accuracy in the far-field). It achieves this by using a blending function to smoothly transition between the two models, making it particularly versatile. A wall-resolving approach was adopted here, meaning no wall functions were used and this typically requires ensuring that the y^+ value of the first cell adjacent to the wall is close to 1 ($y^+ \approx 1$).

Boundary conditions

The accurate definition of boundary conditions is critical for correctly representing the physical problem in a CFD simulation. For all of the simulations, this set of BCs was used for walls, for the front and back patch, respectively for pressure (a) and velocity (b):

```
topWall
{
    type            zeroGradient;
}

bottomWall
{
    type            zeroGradient;
}

front
{
    type            empty;
}

back
{
    type            empty;
}
```

Figure a

```
topWall
{
    type            noSlip;
}

bottomWall
{
    type            noSlip;
}

front
{
    type            empty;
}

back
{
    type            empty;
}
```

Figure b

Obviously different set of BCs were required along the three examined cases. Starting from the **Laminar case** specific pressure(c) and velocity(d) boundary conditions were applied to achieve the desired flow regime with a low Reynolds number:

```
dimensions      [0 2 -2 0 0 0 0];
internalField    uniform 0;
boundaryField
{
    inlet
    {
        type      totalPressure;
        p0         uniform 0.008;
        rho        none;
    }

    outlet
    {
        type      fixedValue;
        value      uniform 0;
    }
}
```

Figure c

```
dimensions      [0 1 -1 0 0 0 0];
internalField    uniform (0 0 0);
boundaryField
{
    inlet
    {
        type      pressureInletOutletVelocity;
        value      uniform (0.06324 0 0);
    }

    outlet
    {
        type      zeroGradient;
    }
}
```

Figure d

It is crucial to note that in OpenFOAM, the pressure variable p is not the absolute or static pressure in physical units (Pascals), while it solves for a modified pressure, often referred to as **kinematic pressure** or "**p-by-rho**", defined as $P_K = P/\rho$, where P is the static pressure (in Pascals) and ρ is the constant fluid density. Consequently, the units of P_K in OpenFOAM are m^2/s^2 and the value is different from the assigned one. For the inlet velocity boundary condition, it was chosen to use the type `pressureInletOutletVelocity` because is a robust choice in OpenFOAM for inlets where a totalPressure is specified, ensuring a prescribed velocity profile.

For the second case, **the turbulent flow case** using the k–ε model, additional boundary conditions were specified for the turbulent quantities, along with updated pressure and velocity values.

```
dimensions      [0 2 -2 0 0 0 0];
internalField    uniform 0;
boundaryField
{
    inlet
    {
        type      totalPressure;
        p0         uniform 107.783;
        rho        none;
    }

    outlet
    {
        type      fixedValue;
        value      uniform 0;
    }
}
```

BCs for pressure

```
dimensions      [0 1 -1 0 0 0 0];
internalField    uniform ( 7.341 0 0);
boundaryField
{
    inlet
    {
        type      pressureInletOutletVelocity;
        value      uniform (7.341 0 0);
    }

    outlet
    {
        type      zeroGradient;
    }
}
```

BCs for velocity

The additional BCs for k and ε consist in application of wall functions near the walls, and in setting a starting estimated value, helpful to reach convergence in easier way. These two formulae were used:

$$k = \frac{3}{2} U^2 I^2 \quad \text{and} \quad \epsilon = 0.09^{0.75} \frac{K^{\frac{3}{2}}}{l_t}$$

where I is the turbulence intensity and l_t the mixing length.

```
dimensions      [0 2 -2 0 0 0 0];
inletk          0.20209;
internalField    uniform $inletk;
boundaryField
{
    topWall
    {
        type      kqRWallFunction;
        value      uniform $inletk;
    }

    bottomWall
    {
        type      kqRWallFunction;
        value      uniform $inletk;
    }

    outlet
    {
        type      zeroGradient;
    }

    inlet
    {
        type      fixedValue;
        value      uniform $inletk;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}
```

BCs for K

```
dimensions      [0 2 -3 0 0 0 0];
inletEpsilon     3.7765;
internalField    uniform 3.7765;
boundaryField
{
    topWall
    {
        type      epsilonWallFunction;
        value      uniform $inletEpsilon;
    }

    bottomWall
    {
        type      epsilonWallFunction;
        value      uniform $inletEpsilon;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }

    outlet
    {
        type      zeroGradient;
    }

    inlet
    {
        type      fixedValue;
        value      uniform $inletEpsilon;
    }
}
```

BCs for ε

For the last case, **the wall-resolved turbulent flow**, using the SST k- ω model, the boundary conditions for pressure, velocity and turbulent kinetic energy are identical to second case one's, but the turbulent quantities now involve ω instead of ϵ . The BCs are set in the same way as did before, and for evaluating the starting value of ω we used:

$$\omega = 0.09^{-0.25} \frac{K^{0.5}}{l_t}$$

```

dimensions      [0 0 -1 0 0 0];
inletOmega      32.83;
internalField    uniform $inletOmega;
boundaryField
{
    topWall
    {
        type      omegaWallFunction;
        value      uniform $inletOmega;
    }
    bottomWall
    {
        type      omegaWallFunction;
        value      uniform $inletOmega;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }

    outlet
    {
        type      zeroGradient;
    }
    inlet
    {
        type      fixedValue;
        value      uniform $inletOmega;
    }
}

```

BCs for omega

Discretization schemes and solvers

The continuous partial differential equations governing the fluid flow were transformed into a system of algebraic equations through spatial discretization. For all conducted simulations (laminar, turbulent with $k-\epsilon$, and turbulent with SST $k-\omega$), a consistent set of discretization schemes and solvers was applied to ensure comparability of results and numerical stability.

```
ddtSchemes
{
    default          steadyState;
}

gradSchemes
{
    default          Gauss linear;
    grad(p)          Gauss linear;
}

divSchemes
{
    default          none;
    div(phi,U)        bounded Gauss linear;
}

laplacianSchemes
{
    default          Gauss linear orthogonal;
}

interpolationSchemes
{
    default          linear;
}

snGradSchemes
{
    default          orthogonal;
}
```

The fvSchemes dictionary for the **first Laminar case** contains:

Time Discretization (ddtSchemes)

The default setting for time derivatives was steadyState: this indicates that the simulations were performed in a steady-state mode, meaning the time derivative terms were either omitted to reach a time-independent solution.

Gradient Schemes (gradSchemes):

For all gradient terms (default) and specifically for the pressure gradient (grad(p)), Gauss linear interpolation was used, which computes gradients using Gaussian integration, that is a second-order accurate central differencing scheme.

Divergence Schemes (divSchemes):

The default divergence scheme was set to none. For the momentum convection term (div(phi,U)), bounded Gauss linear was employed. This is a second-order accurate scheme, where bounded

keyword ensures that the solution remains within physically realistic bounds, preventing overshoots or undershoots that can lead to instability, especially in turbulent flows with sharp gradients.

Laplacian Schemes (laplacianSchemes):

The default scheme for Laplacian terms was Gauss linear orthogonal. This indicates that diffusion terms are discretized using Gaussian integration combined with linear interpolation, specifically for orthogonal meshes.

Interpolation Schemes (interpolationSchemes):

The default interpolation scheme was linear. This is used to interpolate field values from cell centers to cell faces when specific schemes for divergence or Laplacian terms (like Gauss) require face values. It is a second-order accurate interpolation.

Surface Normal Gradient Schemes (snGradSchemes):

The default scheme for surface normal gradients was orthogonal. This implies that the solver assumes a high degree of mesh orthogonality for these terms. While Gauss linear orthogonal for Laplacians provides some robustness to non-orthogonality, this setting suggests that the mesh quality was considered sufficiently orthogonal.

The Navier-Stokes equations involve a strong coupling between the pressure and velocity fields. To resolve this implicit coupling for incompressible steady-state flow, the **SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) algorithm** was employed across all simulation cases (laminar and turbulent). The SIMPLE dictionary shows these settings for the **first Laminar case**:

```
SIMPLE
{
    nNonOrthogonalCorrectors 0;

    residualControl
    {
        p 1e-6;
        U 1e-6;
    }

    relaxationFactors
    {
        fields
        {
            p 0.3;
        }
        equations
        {
            U 0.7;
        }
    }
}
```

- **nNonOrthogonalCorrectors:** Set to 0; this parameter controls the number of additional pressure-velocity correction steps performed to account for mesh non-orthogonality. For meshes with low non-orthogonality, this can be set to 0 as in this case.
- **residualControl for p and U:** The solver monitored the residuals for pressure (p) and velocity (U) equations, with a target residual of 10^{-6} . This criterion indicates the level of convergence required for these fields within each SIMPLE iteration.
- **relaxationFactors for p and U:** for pressure an under-relaxation factor of 0.3 was applied, while for velocity 0.7 was used, to guarantee the optimal condition (the sum of the two should be equal to 1). These factors control the magnitude of changes to the field variables in each iteration, crucial for numerical stability and convergence. Lower factors promote stability, while higher factors accelerate convergence, representing a balance between robustness and speed.

Within each iteration of the outer pressure-velocity coupling algorithm (SIMPLE), large systems of linear algebraic equations must be solved for each primary variable. The fvSolution dictionary defines the settings for linear solvers. These are **the first Laminar case** settings:

Solver for Pressure (p): GAMG (Generalized Algebraic MultiGrid) was used as solver. This is a powerful

```
solvers
{
    p
    {
        solver GAMG;
        smoother GaussSeidel;
        tolerance 1e-08;
        relTol 0;
    }

    U
    {
        solver smoothSolver;
        smoother GaussSeidel;
        tolerance 1e-08;
        relTol 0;
    }
}
```

and efficient multigrid solver commonly employed for pressure equations due to its ability to handle large systems and accelerate convergence by solving on multiple grid levels. GaussSeidel was used as the smoother for the GAMG solver in order to reduce high-frequency errors on the fine grid levels. The internal tolerance for the p solver was set to 10^{-8} , while the relative tolerance was set to 0. This means the absolute tolerance is the primary convergence criterion for the internal pressure solver.

Solver for Velocity (U): smoothSolver was used for solving the velocity components, while GaussSeidel was used as the smoother. The internal tolerance for the U solver was set to 10^{-8} , and the relative tolerance was set to 0.

For the **second case** the Standard $k-\epsilon$ model was used, so several modifications were implemented in the discretization schemes, solver tolerances, and algorithm controls compared to the laminar case. These changes were necessary to accurately capture turbulent phenomena and to ensure numerical stability at higher Reynolds numbers.

Divergence Schemes (divSchemes): The convection term for momentum ($\text{div}(\phi, U)$) was upgraded from bounded Gauss linear to bounded Gauss linearUpwind grad(U). This higher-order scheme provides improved accuracy for convective transport in turbulent flows. New divergence terms for the turbulent kinetic energy ($\text{div}(\phi, k)$) and turbulent dissipation rate ($\text{div}(\phi, \epsilon)$) were introduced, both set to bounded Gauss upwind. This first-order upwind scheme is often chosen for turbulence equations due to its robustness. A new term $\text{div}(\nu_{\text{eff}} \cdot \text{dev2}(T(\text{grad}(U))))$ was added, using Gauss linear.

Laplacian Schemes (laplacianSchemes): The default Laplacian scheme was changed from Gauss linear orthogonal to Gauss linear corrected. The corrected keyword enables a non-orthogonality correction, enhancing robustness on meshes, which is particularly beneficial in turbulent simulations.

```
ddtSchemes
{
    default          steadyState;
}

gradSchemes
{
    default          Gauss linear;
}

divSchemes
{
    default          none;
    div(phi,U)       bounded Gauss linearUpwind grad(U);
    div(phi,epsilon) bounded Gauss upwind;
    div(phi,k)        bounded Gauss upwind;
    div((nuEff*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default          Gauss linear corrected;
}

interpolationSchemes
{
    default          linear;
}

snGradSchemes
{
    default          corrected;
}
```

Surface Normal Gradient Schemes (snGradSchemes): The default snGrad scheme was changed from orthogonal to corrected, providing an explicit non-orthogonality correction for surface normal gradients.

```
SIMPLE
{
    nNonOrthogonalCorrectors 0;

    residualControl
    {
        p          1e-7;
        U          1e-7;
        epsilon    1e-7;
        k          1e-7;
    }
}

relaxationFactors
{
    fields
    {
        p          0.3;
    }
    equations
    {
        U          0.7;
        epsilon    0.6;
        k          0.6;
    }
}
```

While the key differences in Pressure-Velocity coupling, so in the SIMPLE algorithm settings, are:

- The residual Control now explicitly includes k and ϵ alongside p and U . The target residual for all these fields was slightly decreased to 10^{-7} , demanding a higher level of internal convergence within each SIMPLE iteration.
- New relaxation factors were introduced for the turbulent fields: k and ϵ were both set to 0.6. The relaxation factors for p and U remained 0.3 and 0.7 respectively.

```

solvers
{
    p
    {
        solver      GAMG;
        smoother     GaussSeidel;
        tolerance    1e-12;
        relTol       0;
    }

    U
    {
        solver      smoothSolver;
        smoother     GaussSeidel;
        tolerance    1e-10;
        relTol       0;
    }

    "k|epsilon"
    {
        solver      smoothSolver;
        smoother     GaussSeidel;
        tolerance    1e-10;
        relTol       0;
    }
}

```

Analysing now key differences in Linear Solver Settings:

Solver Tolerances: The internal absolute tolerance for the p solver was tightened from 1^{-8} to 1^{-12} . Similarly, the tolerance for the U solver was tightened from 1^{-8} to 1^{-10} . These stricter tolerances demand a more precise solution of the linear systems within each outer iteration.

Turbulence Solvers: New solver entries were added for the turbulent fields: k and epsilon were grouped under "k|epsilon", both using smoothSolver with GaussSeidel smoother, and an absolute tolerance of 1^{-10} .

For **the third wall-resolved turbulent flow** simulation the SST k- ω model was employed, with little specific adjustments to account for the k- ω turbulence model and to reflect the wall-resolved approach:

Divergence Schemes (divSchemes): The primary change here is the replacement of the `div(phi,epsilon)` term with `div(phi,omega)`. Like for the k equation, the convection term for specific dissipation rate (`div(phi,omega)`) was set to bounded Gauss upwind, maintaining the robust first-order scheme for turbulence transport. All other divergence schemes (for U and k) remained consistent with second case.

Wall Distance Calculation (wallDist): A new entry for `wallDist` with method `meshWave` was added: this indicates the use of the `meshWave` algorithm to compute wall distance, which is crucial for the SST k- ω model and particularly relevant for wall-resolved simulations.

```

ddtSchemes
{
    default      steadyState;
}

gradSchemes
{
    default      Gauss linear;
}

divSchemes
{
    default      none;
    div(phi,U)   bounded Gauss linearUpwind grad(U);
    div(phi,omega) bounded Gauss upwind;
    div(phi,k)   bounded Gauss upwind;
    div((nuEff*dev2(T(grad(U))))) Gauss linear;
}

laplacianSchemes
{
    default      Gauss linear corrected;
}

interpolationSchemes
{
    default      linear;
}

snGradSchemes
{
    default      corrected;
}

wallDist
{
    method meshWave;
}

```

While the key differences SIMPLE algorithm settings are:

Non-Orthogonal Correctors

(nNonOrthogonalCorrectors): This parameter was changed from 0 to 1. Introducing one non-orthogonal corrector in the outer SIMPLE loop can enhance stability and accelerate convergence for more complex turbulent flows like those requiring wall-resolution.

Residual Control (residualControl): The field epsilon was replaced by omega in the residual Control block, reflecting the change in the turbulence model. The target residual for all fields (p, U, omega, k) was kept at 10^{-7} , demanding a very strict internal convergence within each SIMPLE iteration, good for a wall-resolved approach.

Relaxation Factors (relaxationFactors): The relaxation factor for the specific dissipation rate (omega) was set to 0.4, replacing the factor for epsilon. The relaxation factors for p, U, and k were adjusted to 0.2, 0.7, and 0.5 respectively.

```
SIMPLE
{
    nNonOrthogonalCorrectors 1;

    residualControl
    {
        p          1e-7;
        U          1e-7;
        omega      1e-7;
        k          1e-7;
    }

    relaxationFactors
    {
        fields
        {
            p          0.2;
        }
        equations
        {
            U          0.7;
            omega      0.4;
            k          0.5;
        }
    }
}
```

```
solvers
{
    p
    {
        solver      GAMG;
        tolerance   1e-07;
        relTol      0;
        smoother    GaussSeidel;
    }

    U
    {
        solver      smoothSolver;
        smoother    symGaussSeidel;
        tolerance   1e-07;
        relTol      0;
    }

    "k|omega"
    {
        solver      GAMG;
        smoother    GaussSeidel;
        tolerance   1e-07;
        relTol      0;
    }
}
```

At least, the key differences in Linear Solver settings are:

Solver Tolerances: The internal absolute tolerance for all primary variables (p, U, k, and omega) was set to 10^{-7} . This represents a significant tightening compared to Case (b)'s 10^{-12} for p and 10^{-10} for U, k, and epsilon. This indicates a potentially different convergence strategy or a less strict requirement for the internal linear solve if the outer loop or overall convergence is robust.

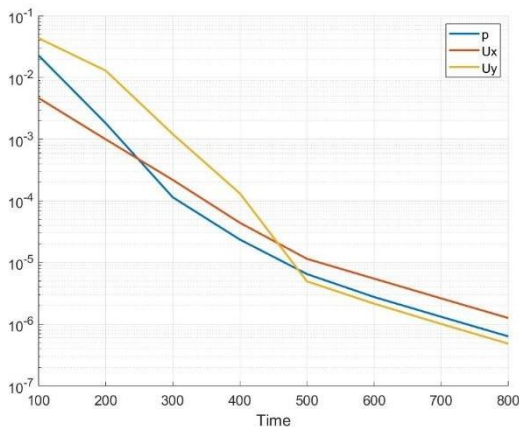
Turbulence Solvers: The solver entry "k|epsilon" was replaced by "k|omega", reflecting the change in the turbulence model. Interestingly, the solver types for "k|omega" was set to GAMG with GaussSeidel smoother, because using GAMG for k and omega can significantly accelerate their convergence in complex turbulent flows, while the p and U solvers rest the same.

Convergence process

After performing the necessary simulations, grid independence has been successfully achieved for both the laminar and turbulent cases; moreover, it is observed that the values of Q and φ remain unchanged for all three simulations carried out on meshes with different qualities.

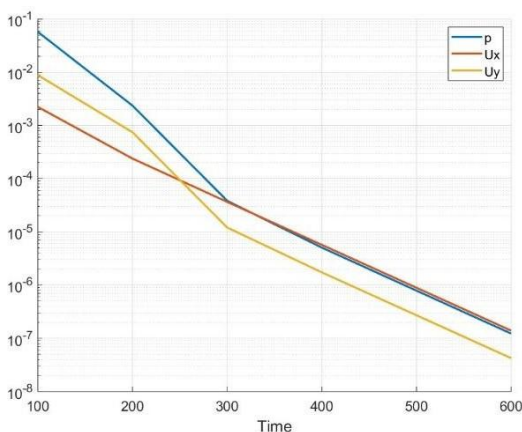
The following table shows the values of the flow rate Q and the reduction coefficient at the nozzle exit φ for each of the three cases required.

	$Q \left[\frac{m^3}{s} \right]$	$\varphi = \frac{V_{OUT}}{V_{OUT,IS}} [-]$
Laminar case	$1.196 \cdot 10^{-4}$	0.7566
Turbulent case with wall functions	0.0175	0.954
Turbulent case without wall functions	0.0175	0.952

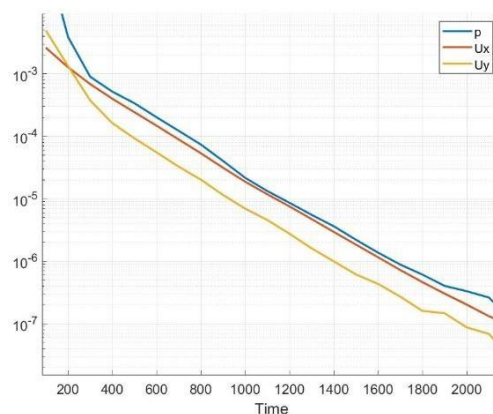


Laminar Case

The convergence process of the CFD simulations was carefully monitored to ensure the reliability of the results. For both cases (laminar and turbulent flow), numerical convergence was verified by monitoring the residuals of the conservation equations (mass, momentum, and energy, if considered, and for the turbulence equations k and ϵ or ω). The residual values were reduced by several orders of magnitude, typically below 10^{-5} or 10^{-6} for the main equations, indicating a stable and consistent numerical solution.



Turbulent case with wall functions



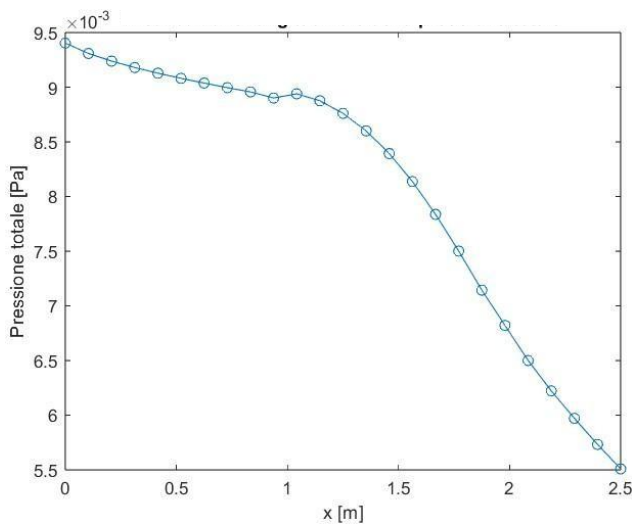
Turbulent case without wall functions

Results and Comparisons

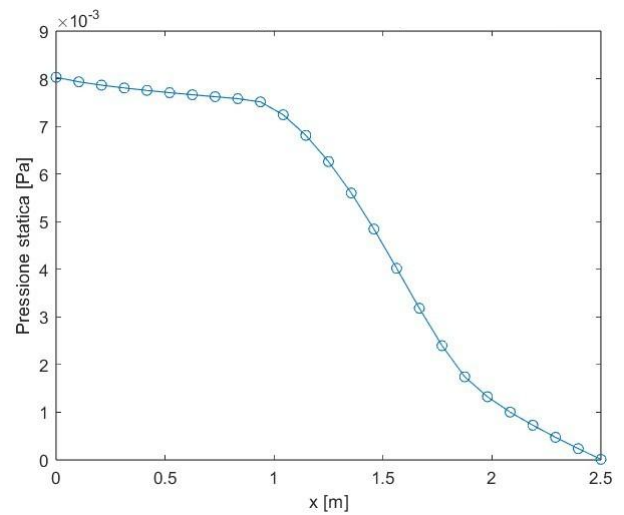
Laminar Case

In laminar flow, viscous effects are predominant, and the boundary layer is thicker, leading to greater energy losses and, consequently, a significantly lower actual velocity compared to the ideal isentropic velocity. The obtained value of $\varphi \approx 0.757$ means that the actual velocity at the outlet is approximately 75% of the ideal velocity, which is expected for a regime where viscous losses are dominant. The associated volumetric flow rate is very small, implying a low Reynolds number, thus confirming the laminar regime.

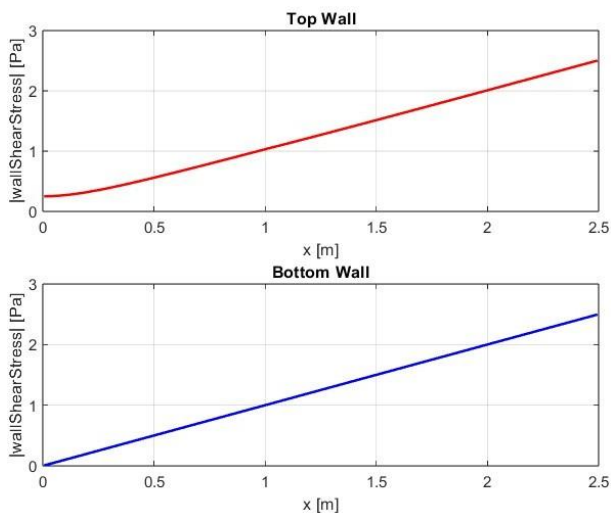
Total pressure distribution



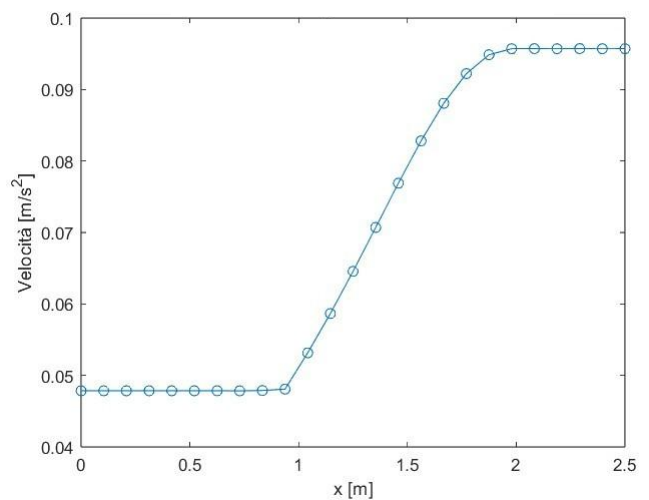
Static pressure distribution



Wall shear stress distribution



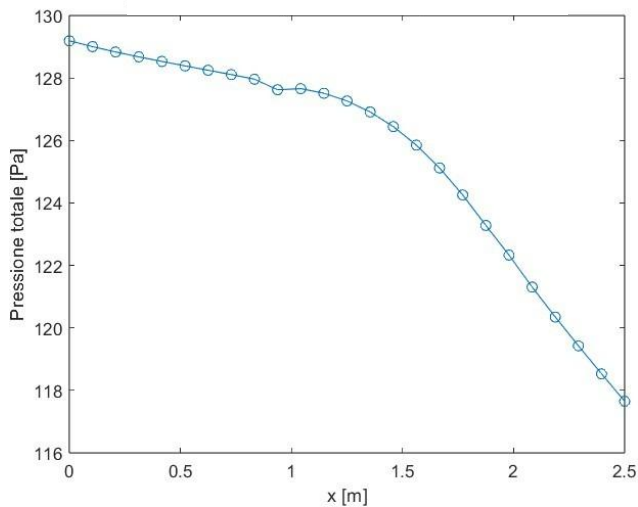
Longitudinal velocity profile



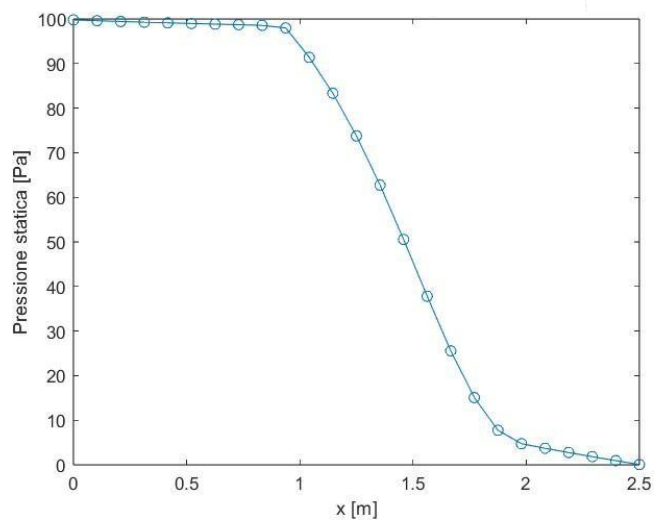
Turbulent Case with Wall-functions

This value of $\phi \approx 0.953$ for the turbulent case was found to be consistent and typical for well-designed converging nozzles or ducts in a turbulent regime. In turbulent flow, the turbulence itself helps to "mix" the flow and maintain a flatter and "fuller" velocity profile across the section, reducing the impact of boundary layer losses on the average velocity compared to laminar flow. Losses due to turbulence do exist, but they are often less significant in proportion to the total kinetic energy compared to viscous losses in laminar flow, especially in accelerating geometries like a converging nozzle. The much higher volumetric flow rate compared to the laminar case confirms the turbulent regime (i.e., a much higher Reynolds number).

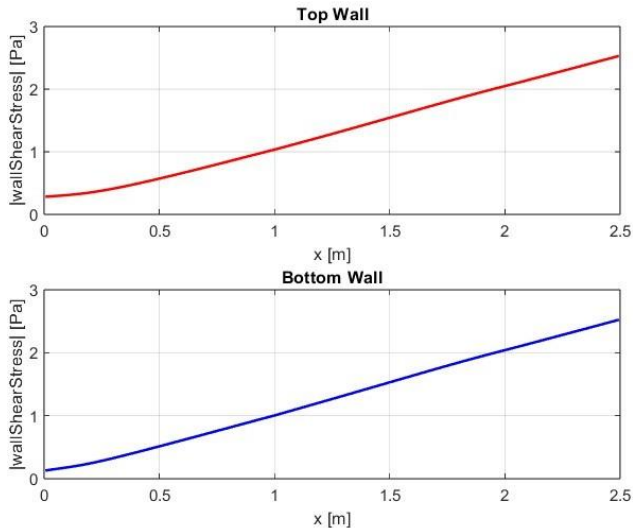
Total pressure distribution



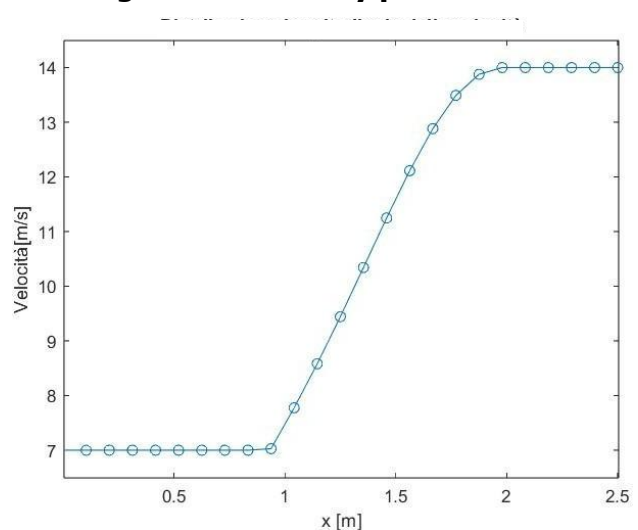
Static pressure distribution



Wall shear stress distribution



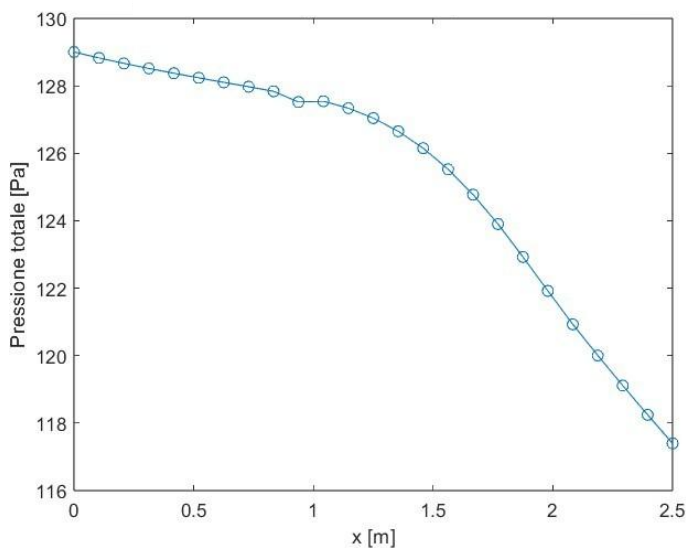
Longitudinal velocity profile



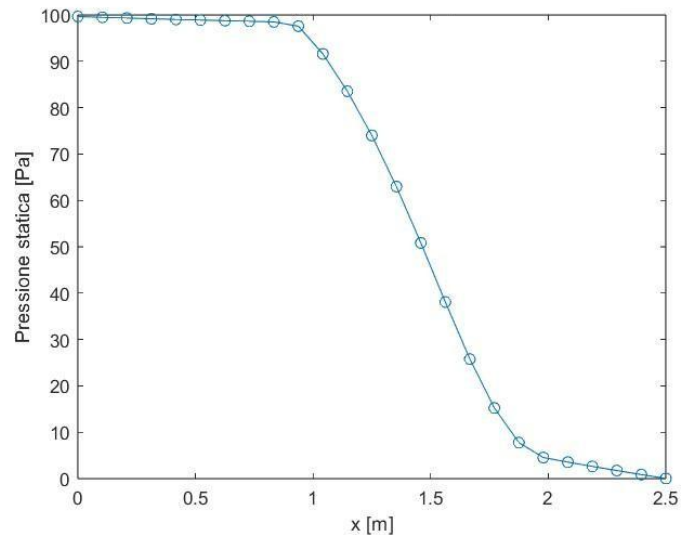
Turbulent case without Wall-functions

For the third case, the results are very close to those of the second, except for a clear difference in the wall shear stress graph. This is a key parameter for understanding the interaction between the flow and the solid surface.

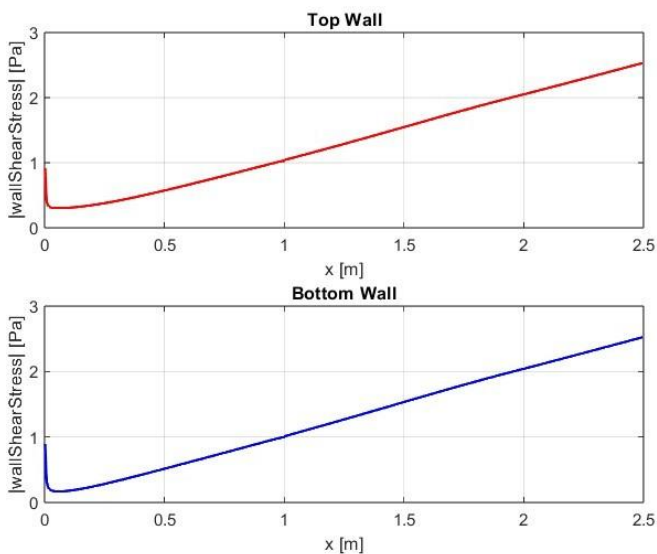
Total pressure distribution



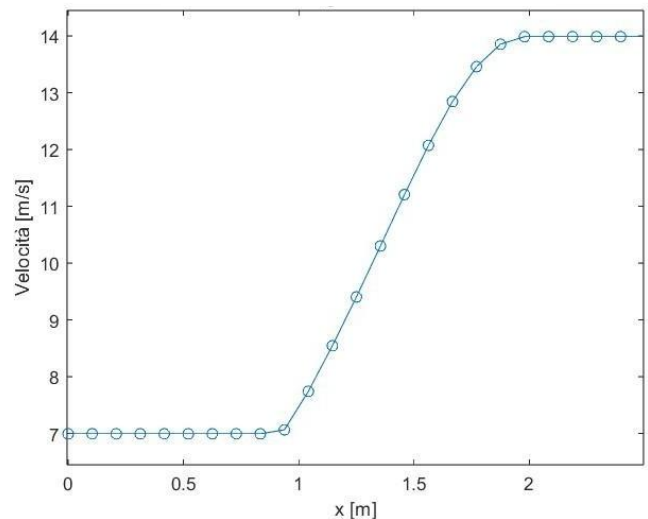
Static pressure distribution



Wall Shear stress distribution



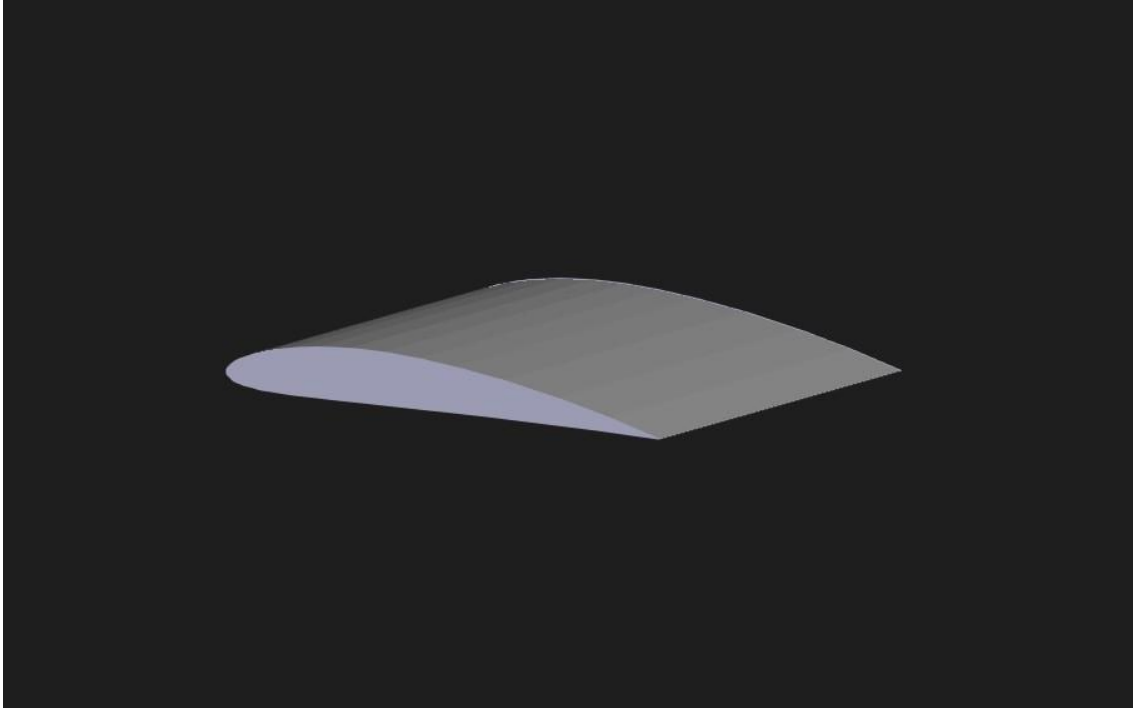
Longitudinal velocity profile



The k-epsilon model's graph shows an almost monotonic increase in shear stress along the entire length of the duct. The value of $|\tau_w|$ starts from an initial minimum and increases progressively as the fluid accelerates in the converging section. As previously stated, it presents significant limitations in modelling the boundary layer near the wall, and the use of "wall functions" can oversimplify the complex physics of the boundary layer in the presence of strong pressure gradients, accelerations, or flow separation, leading to an underestimation or a less precise representation of the shear stress. In our case, the smooth trend without a marked initial peak suggests a potential simplification of the flow's adaptation at the convergence inlet. While the k-omega SST model's graph presents a more complex and, physically, more detailed behaviour. We note a clear peak or a rapid variation in the immediate vicinity of the duct inlet (approximately at $x=0$ m), followed by an increasing trend of shear

stress for the rest of the length, similar but not identical to that of k-epsilon. This occurs because the k-omega SST (Menter's Shear Stress Transport) model is a hybrid model that combines the strengths of the k-omega model near the wall and the k-epsilon model in the free stream. This hybrid formulation gives it greater accuracy in predicting the boundary layer and flows with adverse pressure gradients, separation, and reattachment. The observed behavior with the k-omega SST model, particularly the initial shear stress peak, is physically more plausible for the inlet of a converging duct with turbulent flow. At the entrance of a new geometry or an accelerating section, the flow undergoes a rapid reorganization of the boundary layer, so this can include flow re-adaptation.

Assignment 2 – 2D incompressible flow around the NACA 4412 airfoil



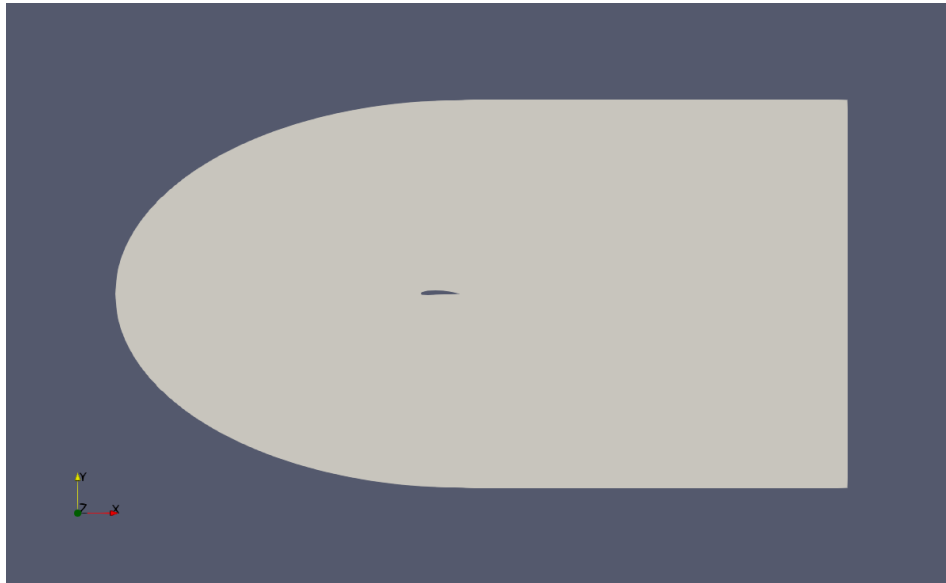
NACA 4412 airfoil- 3D rendering

Abstract:

This report presents the numerical simulation of the two-dimensional incompressible flow around a NACA4412 airfoil with a chord length of 0.1 m, using air at standard conditions ($\nu = 1.5 \cdot 10^{-5} \text{ m}^2/\text{s}$, $\rho = 1.2 \text{ kg/m}^3$). The boundary conditions include an inlet velocity of 20 m/s, zero angle of attack, and ambient pressure of 1 bar. Turbulence was modeled using the Spalart-Allmaras (SA) and $k-\epsilon$ models with wall functions, conducting a mesh independence study through three levels of mesh refinement. Additionally, a simulation was performed using the SA model with near-wall resolution, avoiding the use of wall functions. Aerodynamic performance was evaluated by lift and drag coefficients, pressure and shear stress distributions on the airfoil surfaces, and velocity distribution in the wake one chord length downstream of the trailing edge. The report provides a detailed comparison between the modeling approaches and their impact on the prediction of relevant flow quantities.

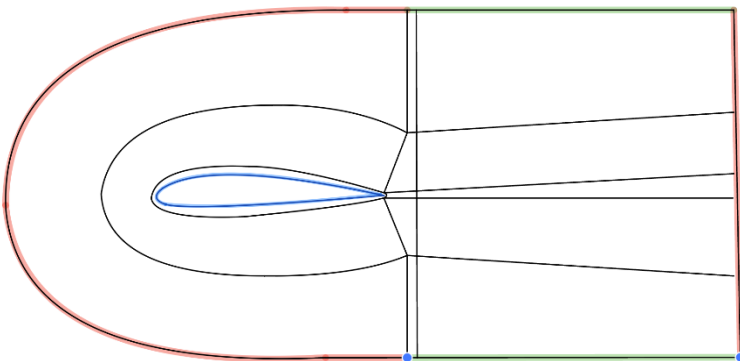
Geometrical domain

We are working on a 2D case of an incompressible flow which is running around a NACA4412 airfoil.



For what concern the dimensions of the geometrical domain, the whole flow region has been expanded around the airfoil for distances far enough to guarantee the following properties:

- 7 chords ahead to guarantee an undisturbed flow at the inlet.
- 5 chords upwards and downwards to avoid the influence of the boundaries on the fluid circulation around the airfoil.
- 10 chords behind to have a better representation of the wake



- Red surfaces: patch boundary type
- Green surfaces: symmetry boundary type
- Lateral surfaces: empty boundary type (left and right)

```
sx
{
    type        empty;
}

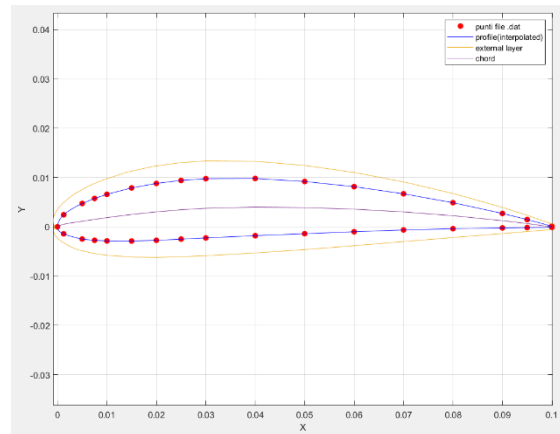
dx
{
    type        empty;
}

top
{
    type        symmetry;
}

bottom
{
    type        symmetry;
}
```

Meshes layout

It was decided to adopt a hybrid mesh: a C type mesh was built for the external inlet region, while around the airfoil, an O type mesh is able to cope the curvature of its shape: the O mesh also allows to control better the refinements inside the boundary layer and select the distance from the wall, whenever the usage of wall functions requires appropriate values of y^+ .



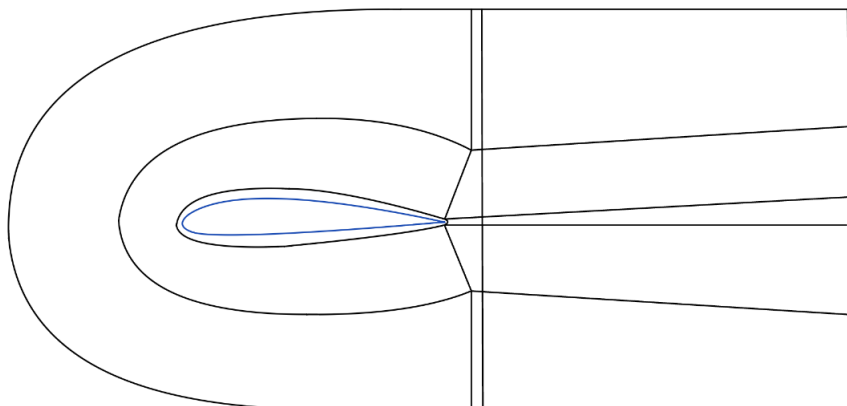
The O type mesh which wraps the profile has been created following this reasoning: after importing the .dat file of the airfoil profile, an interpolation of the airfoil itself has been performed to increase the number of points, especially near the leading edge, to smooth the curvature.

Then, to compute the external layer of the region, the normal unit vectors to the surface have been multiplied by a fixed value plus a contribution dependent on the airfoil thickness. The thickness was defined as the distance between the suction side and the pressure side relative to the chord line.

An additional intermediate level between the C and O type regions was added to have a better control on the cell-to-cell expansion ratio.

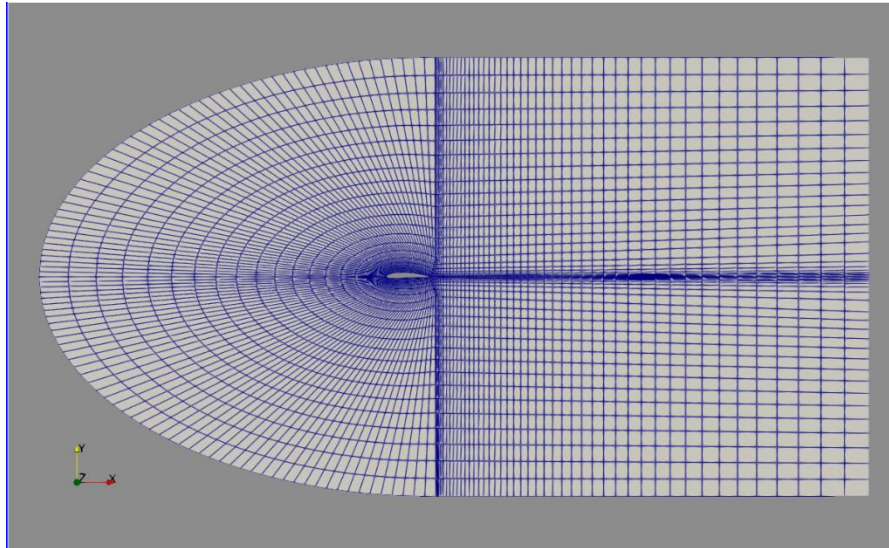
Near the trailing edge a sequence of blocks was added to better absorb the cell distortion caused by the choice of closing the trailing edge itself with a rounded end.

For the outlet region, the domain was completed with a set of rectangular sub-blocks cells with a diverging behaviour for the middle ones in order to better represent the wake.



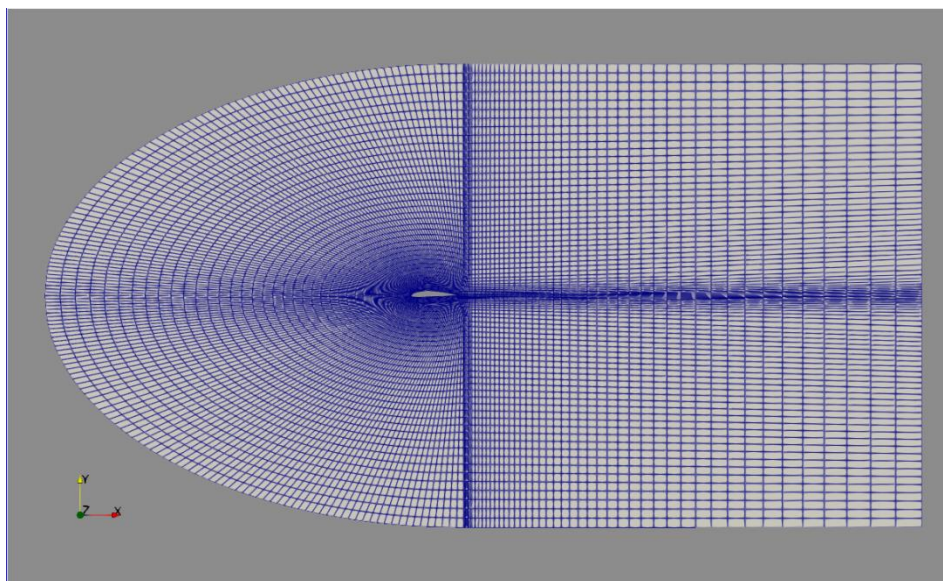
The assignment required to make a grid independence study by using turbulent models with wall functions. The study was conducted on three different levels of refining meshes, gradually increasing the cells number and working on the boundary layer region to guarantee always the right values of y^+ near the walls:

Coarse Mesh



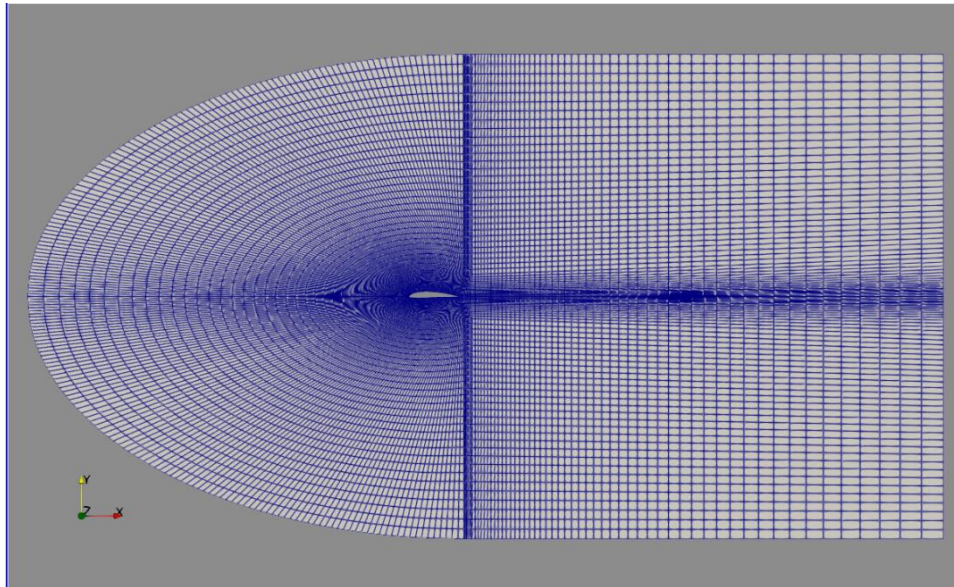
N° of cells = 8676

Medium Mesh

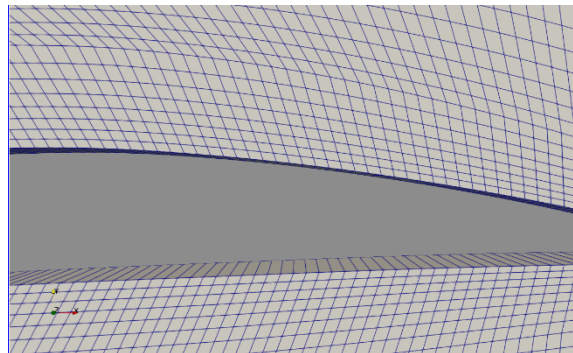


N° of cells = 11808

Fine Mesh



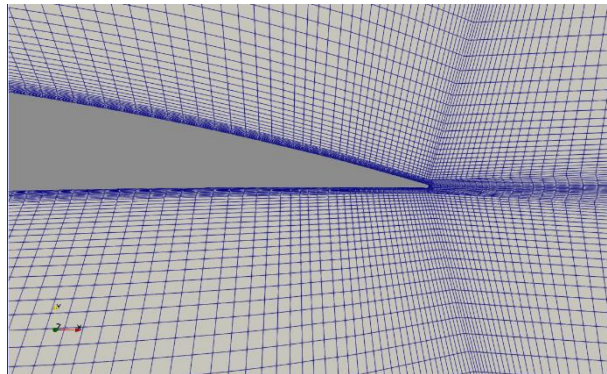
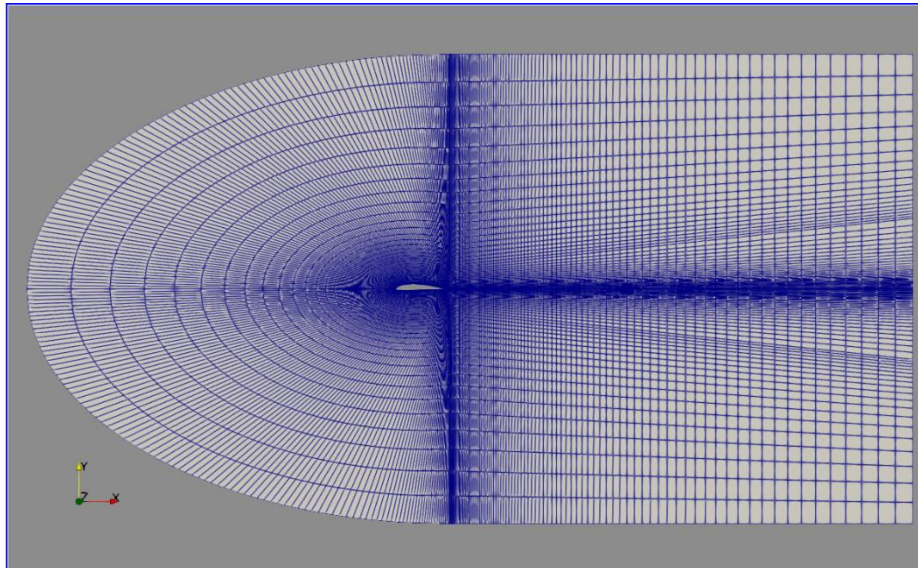
N° of cells = 16838



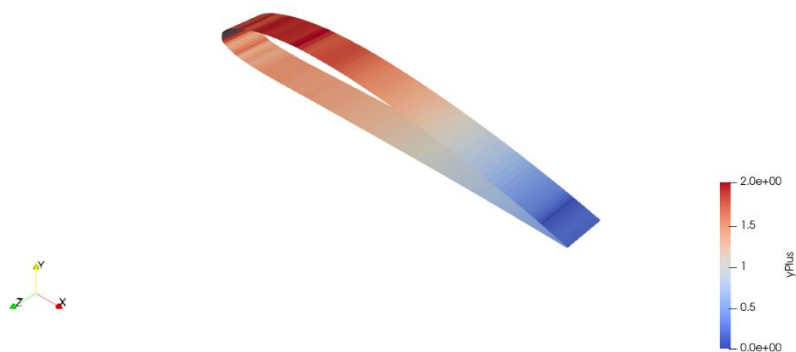
y+ distribution along the profile

Wall Mesh

For the last case, it was required to perform a turbulent simulation without the usage of wall functions; the number of cells and the grading was carefully changed to have proper values of y^+ in correspondence of the walls and guarantee also a good mesh refinement.



N° of cells = 26500



y+ distribution along the profile

Flow model and boundary conditions

Flow model

Spalart-Allmaras (SA) Model

The Spalart-Allmaras model is a single-equation transport model that solves for a modified turbulent kinematic viscosity. It is particularly favoured in aerodynamics for its robustness and its ability to effectively predict flows with adverse pressure gradients and separation.

- **Implementation with Wall Functions:** For simulations utilizing wall functions (as required in point (a) of the exercise), the SA model employs empirical functions to model flow behavior in the near-wall region. This approach significantly reduces the mesh refinement required within the viscous sublayer, optimizing computational time.
- **Implementation without Wall Functions:** For the last simulation required the SA model was used to directly resolve the flow up to the wall, thus avoiding the use of wall functions. This methodology demands extreme mesh refinement in the vicinity of the airfoil surface (with the first mesh point located at $y^+ < 1$) to accurately capture the details of the viscous sublayer. Although computationally more expensive, this configuration provides a more faithful description of the near-wall flow physics and viscous drag.

k- ϵ Model

The k- ϵ model follows the same criteria as explained for the first assignment; so essentially wall functions were required, and all other hypothesis are the same.

Boundary conditions

For the three different cases it was necessary to set different boundary conditions, that, except for the variation of turbulent viscosity ν_t , kept constant for S.A. and for k-epsilon, obviously taking in account the clear differences between the methods. Starting from the **first simulation for SA**, for grid independence study, these were the settings:

```
dimensions      [0 2 -2 0 0 0 0];
internalField    uniform 0;
boundaryField
{
    inlet
    {
        type      zeroGradient;
    }
    outlet
    {
        type      fixedValue;
        value      uniform 0;
    }
    profile
    {
        type      zeroGradient;
    }
}
```

BCs for pressure

```
dimensions      [0 1 -1 0 0 0 0];
internalField    uniform ( 20 0 0 );
boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform (20 0 0);
    }
    outlet
    {
        type      zeroGradient;
    }
    profile
    {
        type      noSlip;
    }
}
```

BCs for velocity

```

dimensions      [0 2 -1 0 0 0 0];
internalField    uniform 0.000134;
boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0.000134;
    }

    outlet
    {
        type      zeroGradient;
    }

    profile
    {
        type      nutUSpaldingWallFunction;
        value      uniform 0;
    }
}

```

BCs for turbulent viscosity *nut*

```

dimensions      [0 2 -1 0 0 0 0];
internalField    uniform 0.000134;
boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 0.000134;
    }

    outlet
    {
        type      zeroGradient;
    }

    profile
    {
        type      fixedValue;
        value      uniform 0;
    }
}

```

BCs for *nuTilda*

$\mu_t = 0.09 \frac{k^2}{\epsilon}$ used to evaluate the value at inlet.

When using the Spalart-Allmaras turbulence model with wall functions in OpenFOAM, the key difference in boundary condition settings for μ_t (eddy viscosity) and *nuTilda* (the transport variable in SA model) lies at the wall boundaries:

- **nut (Kinematic Eddy Viscosity):** at walls, μ_t uses a specific wall function, that dynamically calculates the eddy viscosity at the cell centers adjacent to the wall, accounting for the near-wall flow physics without requiring a very fine mesh in the viscous sublayer. The initial value is set uniformly to 0, because this is used as an initial guess by the wall function itself.
- **nuTilda (Spalart-Allmaras Transport Variable):** at walls, *nuTilda* is typically set to a *fixedValue* of 0, reflecting the physical condition that the turbulent production variable approaches zero directly at the solid surface.

For the **second simulation** it was used a $k - \epsilon$ model, for which the same settings for pressure and velocity were kept, and BCs for turbulent kinetic energy and dissipation rate required the usage of wall functions, due to model's hypothesis. Finally, boundaries were posed in this way:

```
dimensions      [0 2 -2 0 0 0 0];
inletk          0.06;
internalField   uniform $inletk;
boundaryField
{
    profile
    {
        type      kqRWallFunction;
        value      uniform $inletk;
    }
    sx
    {
        type      empty;
    }
    dx
    {
        type      empty;
    }
    outlet
    {
        type      zeroGradient;
    }
    inlet
    {
        type      fixedValue;
        value      uniform $inletk;
    }
    top
    {
        type      symmetry;
    }
    bottom
    {
        type      symmetry;
    }
}
```

BCs for K

```
dimensions      [0 2 -3 0 0 0 0];
inletEpsilon    2.415;
internalField    uniform 2.415;    //$inletEpsilon;
boundaryField
{
    profile
    {
        type      epsilonWallFunction;
        value      uniform $inletEpsilon;
    }
    sx
    {
        type      empty;
    }
    dx
    {
        type      empty;
    }
    outlet
    {
        type      zeroGradient;
    }
    inlet
    {
        type      fixedValue;
        value      uniform $inletEpsilon;
    }
    top
    {
        type      symmetry;
    }
    bottom
    {
        type      symmetry;
    }
}
```

BCs for ϵ

The **last simulation** was performed using **Spalart-Allmaras model**, but without the help of wall functions; a much more refined mesh was required, to solve properly the flow along the profile. The only boundary condition that was modified is the one for turbulent viscosity μ_t .

```
dimensions      [0 2 -1 0 0 0 0];
internalField    uniform 0.000134;
boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0.000134;
    }
    outlet
    {
        type      zeroGradient;
    }
    profile
    {
        type      calculated;
        value      uniform 0;
    }
}
```

Discretization schemes and solvers

To simplify the equations in a set of linear algebraic ones, the following discretization procedures, solver tolerances and algorithm controls have been implemented to better represent the turbulent phenomena and to ensure numerical stability.

Spalart-Allmaras:

The discretization schemes don't vary with respect of the ones used for turbulent simulations in the first assignment, except for the new variable *nuTilda* introduced, that needs to be discretized setting a proper *divergence scheme*:

The convection term scheme chosen for *nuTilda* ($\text{div}(\phi, \text{nuTilda})$) is bounded Gauss linearUpwind grad(U). This second-order scheme provides good accuracy for convective transport in turbulent flows.

```
ddtSchemes
{
    default          steadyState;
}

gradSchemes
{
    default          Gauss linear;
}

divSchemes
{
    default          none;
    div(phi,U)       bounded Gauss linearUpwind grad(U);
    div(phi,nuTilda) bounded Gauss linearUpwind grad(nuTilda);
    div((nuEff*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default          Gauss linear corrected;
}

interpolationSchemes
{
    default          linear;
}

snGradSchemes
{
    default          corrected;
}
```

fvSchemes

For what concerns with the chosen solvers, literally nothing has changed from the settings of previous assignment, because a SIMPLE algorithm was used as before. The only difference is in adding the definition of *nuTilda* parameter.

```
SIMPLE
{
    nNonOrthogonalCorrectors 0;

    residualControl
    {
        p          1e-5;
        U          1e-5;
        nuTilda    1e-5;
    }

    relaxationFactors
    {
        fields
        {
            p          0.3;
        }
        equations
        {
            U          0.7;
            nuTilda    0.3;
        }
    }
}
```

```
solvers
{
    p
    {
        solver      GAMG;
        tolerance   1e-08;
        relTol      0;
        smoother    GaussSeidel;
    }

    U
    {
        solver      smoothSolver;
        smoother    GaussSeidel;
        tolerance   1e-08;
        relTol      0;
    }

    nuTilda
    {
        solver      smoothSolver;
        smoother    GaussSeidel;
        tolerance   1e-08;
        relTol      0;
    }
}
```

K- ϵ model

Even in this case, everything said for this model, used in the previous assignment, don't change at all.

```
ddtSchemes
{
    default          steadyState;
}

gradSchemes
{
    default          Gauss linear;
}

divSchemes
{
    default          none;
    div(phi,U)       bounded Gauss linearUpwind grad(U);
    div(phi,epsilon) bounded Gauss upwind;
    div(phi,k)        bounded Gauss upwind;
    div((nuEff*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default          Gauss linear corrected;
}

interpolationSchemes
{
    default          linear;
}

snGradSchemes
{
    default          corrected;
}
```

```
SIMPLE
{
    nNonOrthogonalCorrectors 0;

    residualControl
    {
        p          1e-5;
        U          1e-5;
        epsilon    1e-5;
        k          1e-5;
    }

    relaxationFactors
    {
        fields
        {
            p          0.3;
        }
        equations
        {
            U          0.7;
            epsilon    0.6;
            k          0.6;
        }
    }
}
```

```
solvers
{
    p
    {
        solver      GAMG;
        smoother    GaussSeidel;
        tolerance    1e-12;
        relTol      0;
    }

    U
    {
        solver      smoothSolver;
        smoother    GaussSeidel;
        tolerance    1e-10;
        relTol      0;
    }

    "k|epsilon"
    {
        solver      smoothSolver;
        smoother    GaussSeidel;
        tolerance    1e-10;
        relTol      0;
    }
}
```

Convergence and results:

The grid independence study produces the following values for the Drag and Lift coefficients:

k-epsilon (with Wall-functions)	N° of cells	$C_D[-]$	$C_L[-]$
Coarse Mesh	8676	0.021	0.3756
Medium Mesh	11808	0.0202	0.3790
Fine Mesh	16838	0.0198	0.3898

S-A (with Wall-functions)	N° of cells	$C_D[-]$	$C_L[-]$
Coarse Mesh	8676	0.0227	0.3742
Medium Mesh	11808	0.0222	0.3835
Fine Mesh	16838	0.0219	0.3906

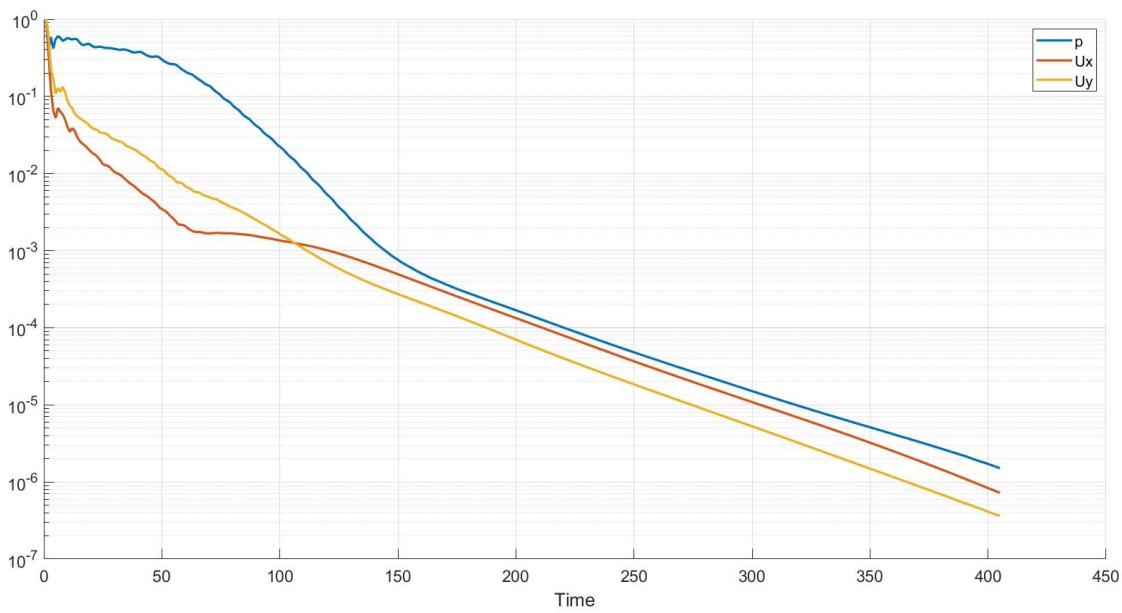
By looking at the tables, it is possible to confirm that the grid independence has been successfully achieved, as by comparing the C_d and C_l values between the coarsest and the finest meshes the differences can be accepted ($\sim 3\%$).

For this reason, the plots required for the first part of the assignment are all referred to the last simulation performed with the finest mesh among the three.

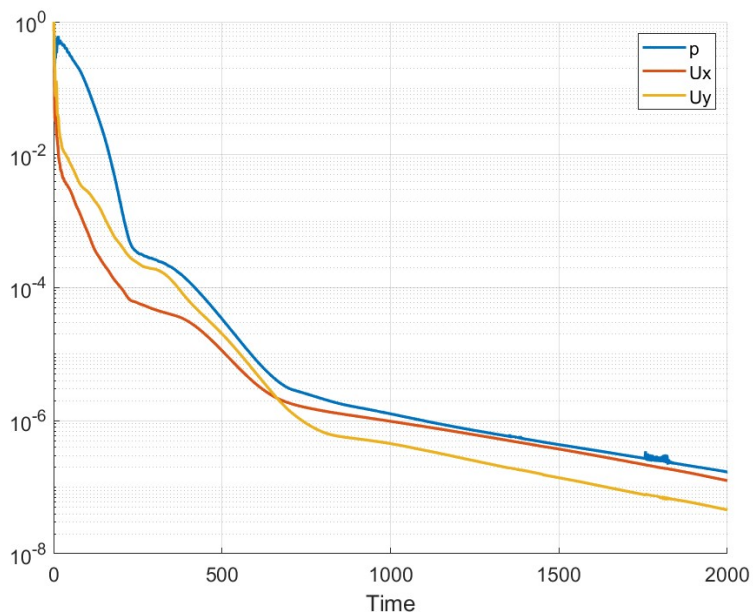
The last simulation performed with S-A models and resolving directly up to the wall gives the following values:

S-A (without Wall-functions)	N° of cells	$C_D[-]$	$C_L[-]$
Wall Mesh	26500	0.0203	0.3914

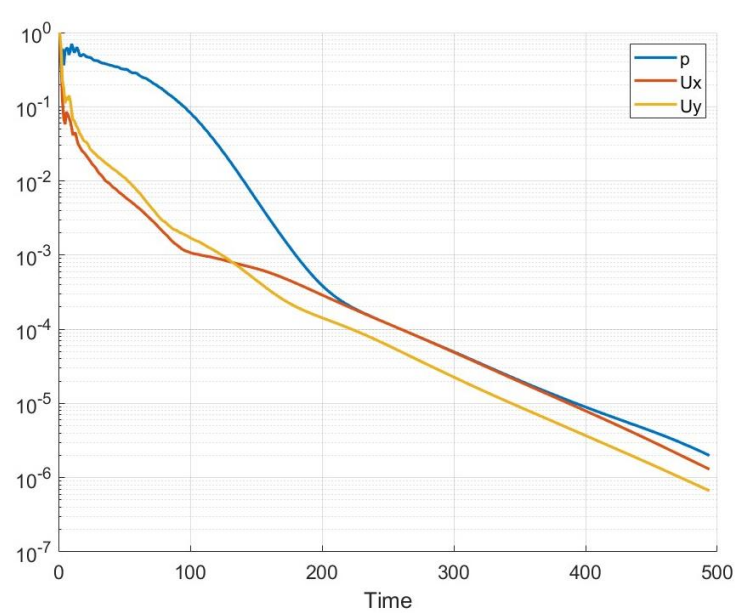
Residuals plots



k-epsilon (with Wall-functions)



S-A (without Wall-functions)

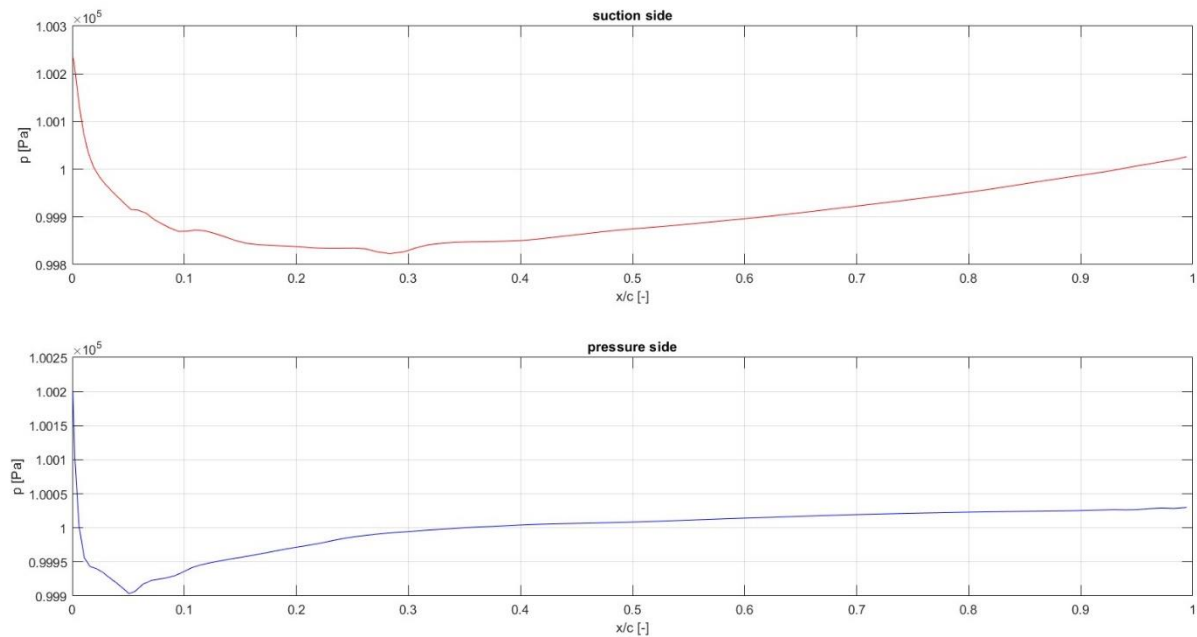


S-A (with Wall-functions)

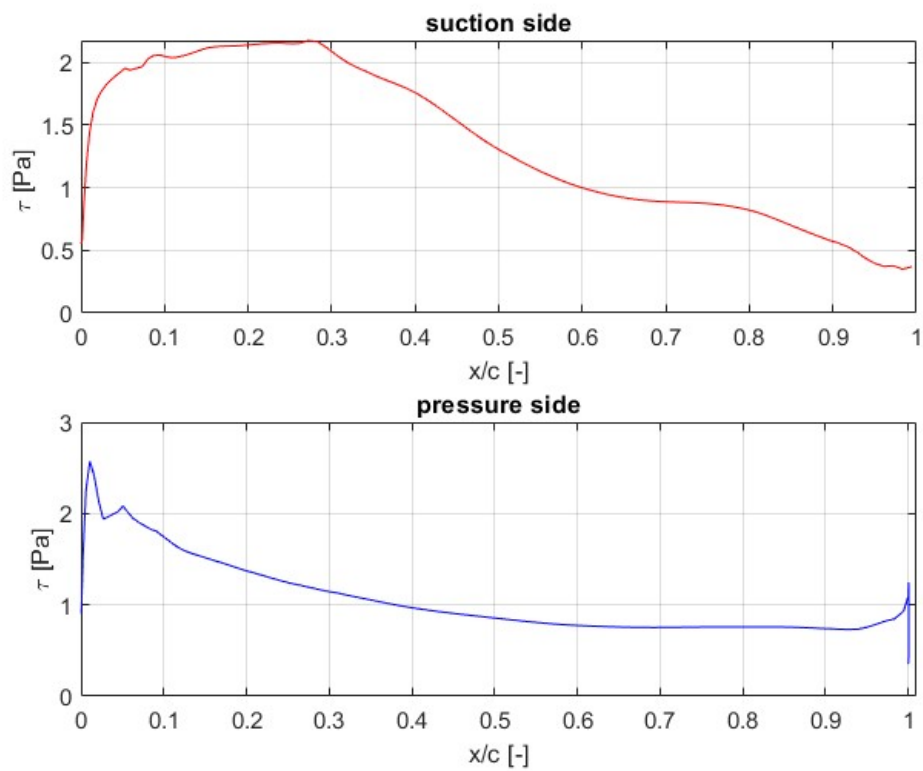
Results and Comparisons

k-epsilon (with Wall-functions)

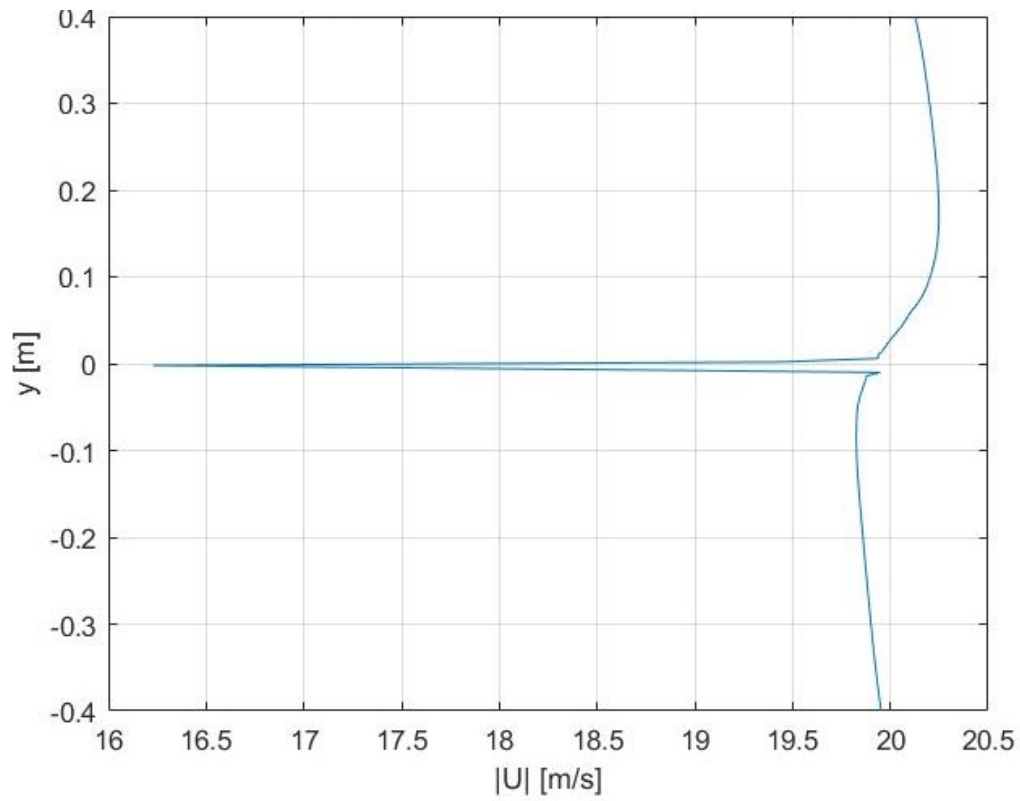
Pressure distribution



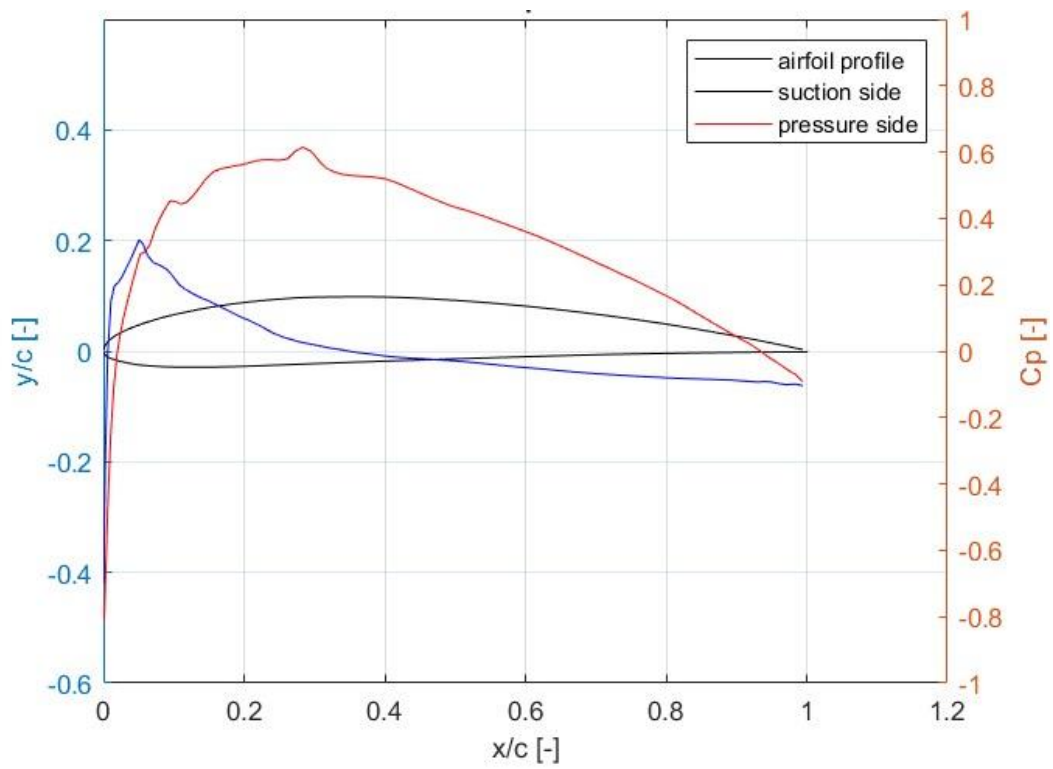
Wall shear stresses distribution



Velocity distribution in the wake (1 chord down to trailing edge)

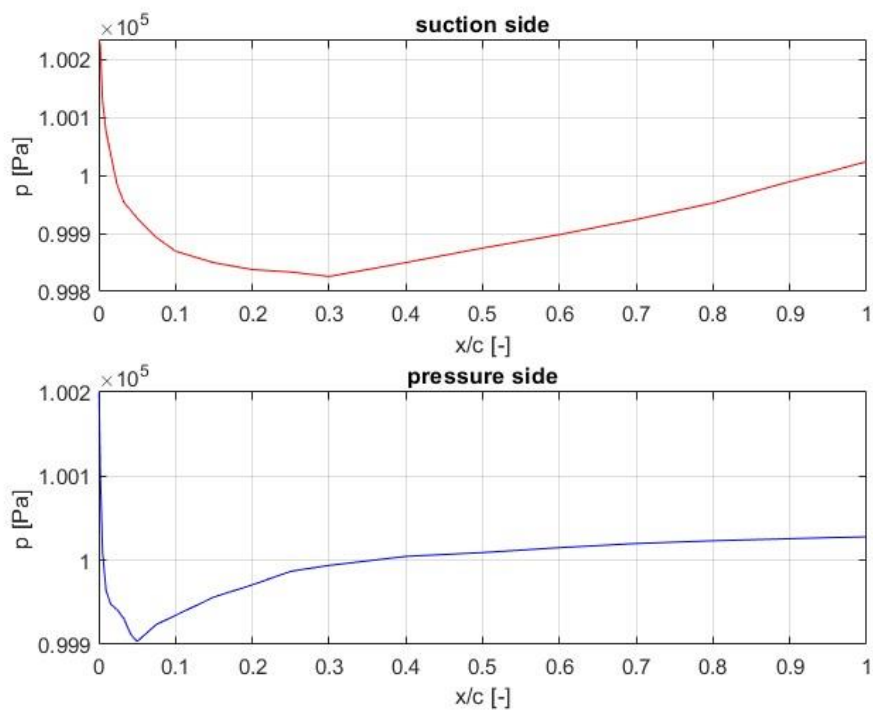


Cp distribution

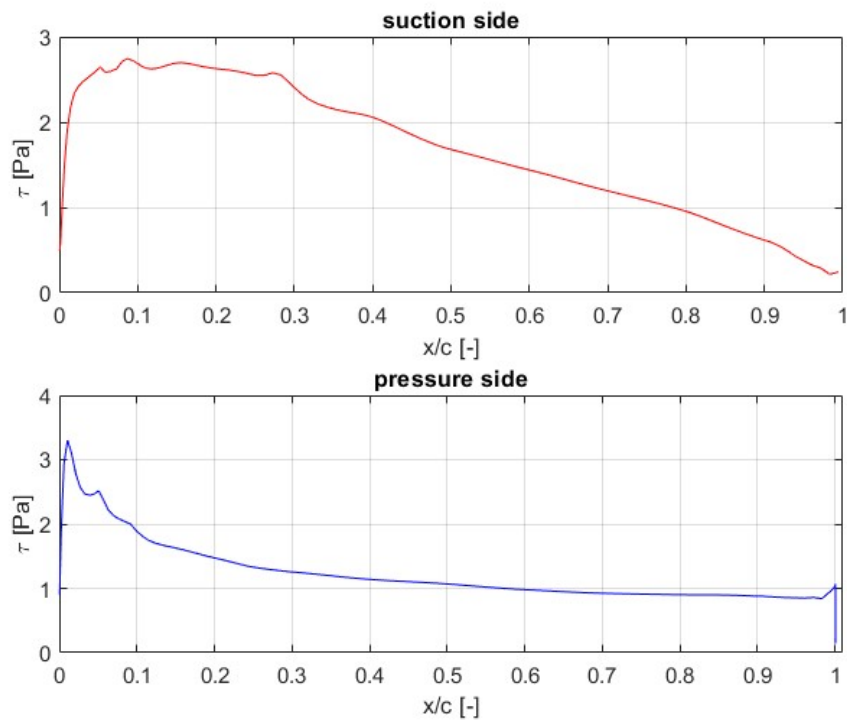


S-A (with Wall-functions)

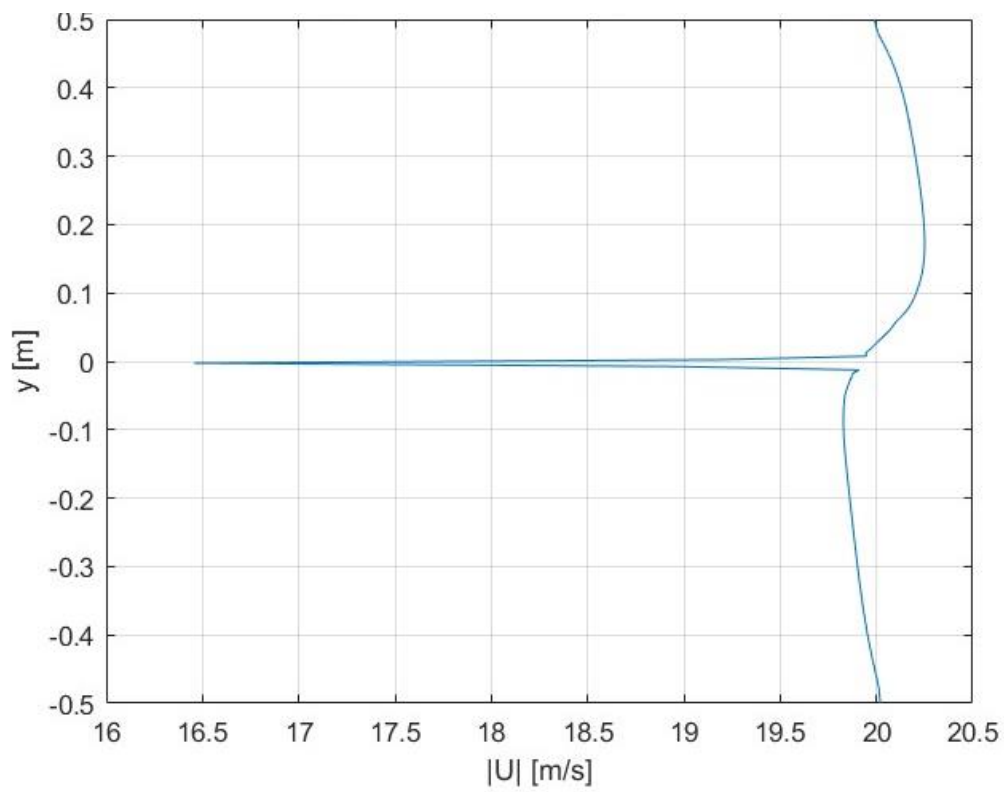
Pressure distribution



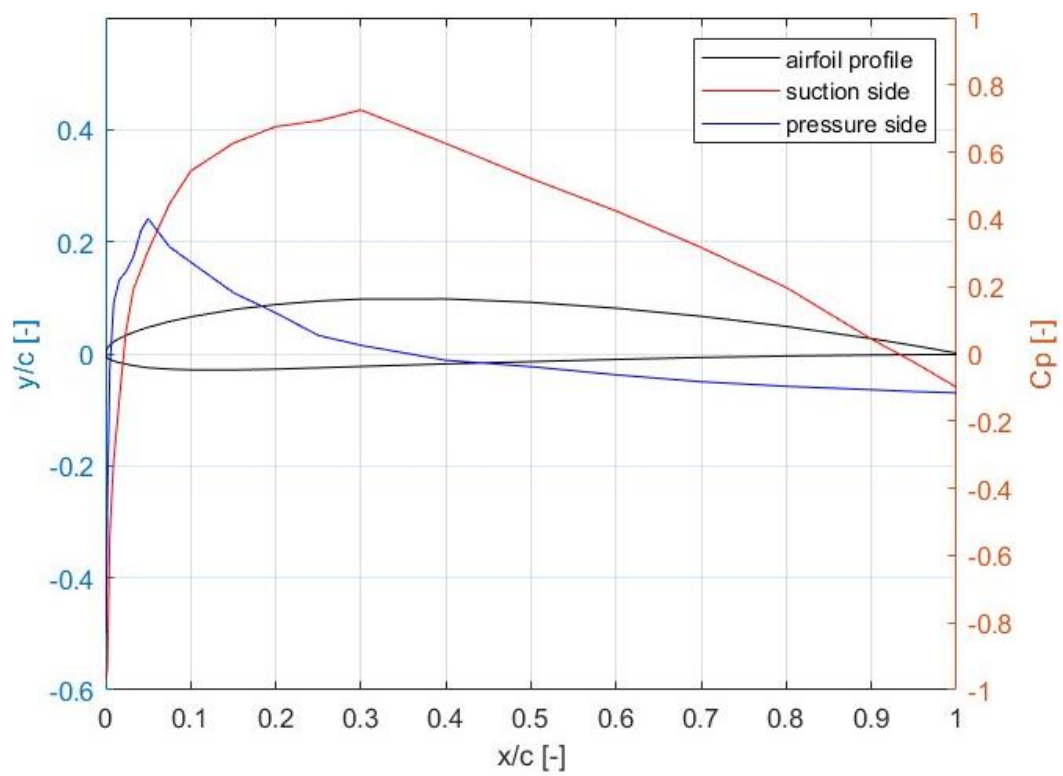
Wall shear stresses distribution



Velocity distribution in the wake (1 chord down to trailing edge)

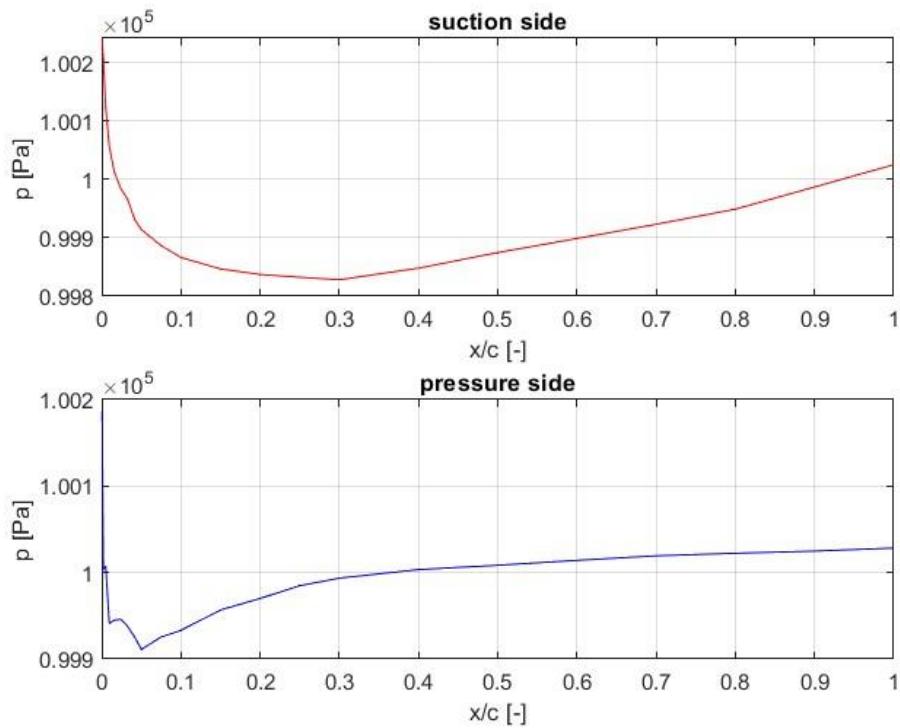


Cp distribution

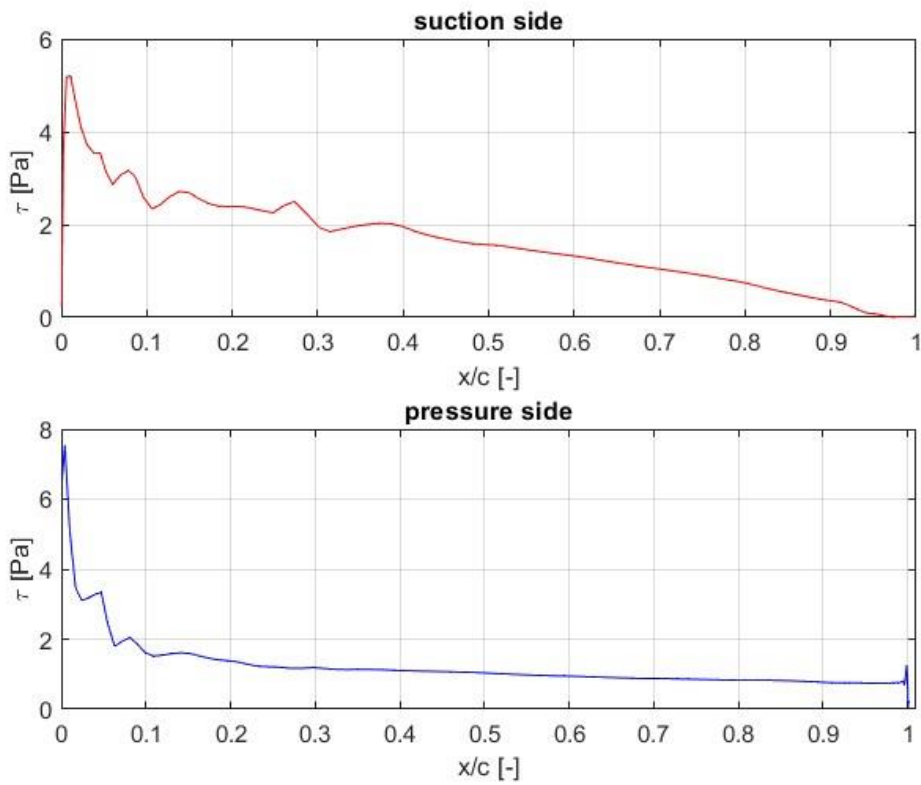


S-A (without Wall-functions)

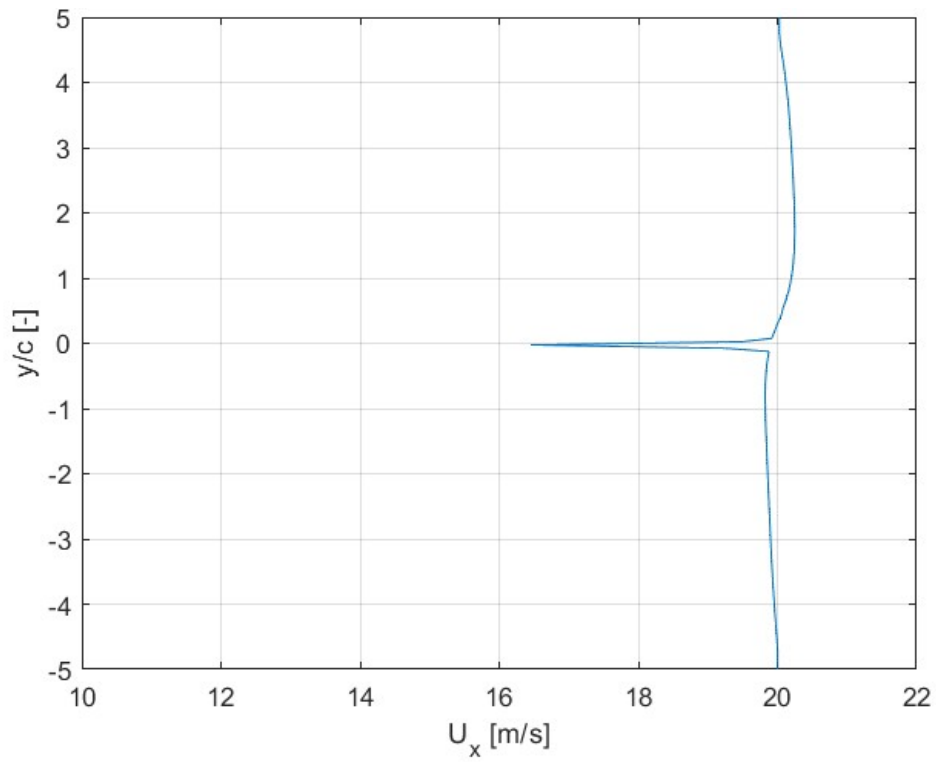
Pressure distribution



Wall shear stresses distribution



Velocity distribution in the wake (1 chord down to trailing edge)



Cp distribution

